LAPORAN TUGAS BESAR 1
IF3070 DASAR INTELEGENSI ARTIFISIAL

PENCARIAN SOLUSI PENEMPATAN KENDARAAN PADA GELADAK KAPAL DENGAN LOCAL SEARCH



Disusun oleh Kelompok 10 :

Indanna Halwah Raqiqah Masyhur	18224014
Evan Nathanael Tanuri	18224019
Muhammad Okto Huzainy	18224021

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

DAFTAR ISI

DAFTAR ISI **2**
1. DESKRIPSI PERSOALAN **3**
2. PEMBAHASAN **4**
2.1. Objective Function 4
2.1.1. Tujuan 4
2.1.2. Batasan 4
2.1.3. Rumus 4
2.1.4. Heuristik 4
2.2. Implementasi Algoritma Local Search 5
2.2.1. Simple Hill-Climbing 5
2.2.2. Hill-Climbing Steepest Ascent 7
2.2.3. Stochastic Hill Climbing 8
2.2.4. Hill-Climbing with Sideways Move 10
2.2.5. Random Restart Hill Climbing 12
2.2.6. Simulated Annealing 14
2.2.7. Genetic Algorithm 19
2.3. Hasil dan Analisis 23
2.3.1. Hasil Eksperimen 23
2.3.1.1. Simple Hill Climbing 23
2.3.1.2. Hill-Climbing Steepest Ascent 25
2.3.1.3. Stochastic Hill Climbing 28
2.3.1.4. Hill-Climbing with Sideways Move 30
2.3.1.5. Random Restart Hill Climbing 33
2.3.1.6. Simulated Annealing 36
2.3.1.7. Genetic Algorithm 42
2.3.2. Hasil Analisis 58
3. KESIMPULAN DAN SARAN **61**
3.1. Kesimpulan 61
3.2. Saran 61
4. PEMBAGIAN TUGAS **62**
5. REFERENSI **63**
6. LAMPIRAN **64**

1. DESKRIPSI PERSOALAN

Persoalan ini adalah penentuan kendaraan yang diangkut serta posisi dan orientasinya pada geladak kapal kargo agar total tarif pengiriman (*ShippingFee*) maksimal. Geladak dan kendaraan direpresentasikan sebagai grid dua dimensi. Kendaraan dapat berbeda ukuran, dan tidak semua kendaraan harus diangkut. Setiap kendaraan memiliki dimensi, orientasi (horizontal atau vertikal), *ShippingFee*, berat, dan ETA (perkiraan waktu kedatangan dalam sisa hari), sedangkan kapal memiliki dimensi geladak dan kapasitas angkut maksimum (*MaxCapacity*).

Sebuah solusi valid apabila kendaraan tidak saling bertumpuk, seluruh sel kendaraan berada di dalam batas geladak, dan total berat kendaraan yang diangkut tidak melebihi *MaxCapacity*. Pencarian dilakukan dengan *local search*, dimulai dari *initial state* acak, lalu berpindah antar *neighbor* melalui *swap*, *relocate*, dan *rotate*. Algoritma yang diimplementasikan adalah *Hill-Climbing Steepest Ascent*, *Stochastic Hill Climbing*, *Hill-Climbing with Sideways Move*, *Random Restart Hill-Climbing*, *Simulated Annealing*, dan *Genetic Algorithm*.

2. PEMBAHASAN

2.1. Objective Function

2.1.1. Tujuan

Persoalan ini bertujuan memaksimalkan *ShippingFee* kendaraan yang diangkut oleh kapal. Setiap kendaraan boleh diangkut atau ditinggalkan di luar kapal, sehingga solusi menentukan kendaraan mana yang diangkut beserta posisi dan orientasinya di dalam geladak.

2.1.2. Batasan

Sebuah state dianggap valid apabila memenuhi seluruh batasan berikut:

1. Kendaraan tidak saling bertumpuk, yaitu tidak ada sel grid geladak yang ditempati lebih dari satu kendaraan.
2. Seluruh sel grid yang ditempati suatu kendaraan berada di dalam batas geladak kapal.
3. Total berat kendaraan pada tiap kapal tidak melebihi MaxCapacity kapal tersebut.

Seluruh state yang dibangkitkan, baik initial state, neighbor, maupun anak pada Genetic Algorithm, selalu memenuhi ketiga batasan ini, sehingga nilai objective function hanya dihitung untuk state yang valid.

2.1.3. Rumus

Nilai objective function (fitness) sebuah state *s* dihitung sebagai berikut :

$$f(S) = \sum_{v \in V_{\text{terpilih}}} \text{ShippingFee}(v) - \alpha \sum_{v \in V_{\text{terpilih}}} \text{ETA}(v)$$

Dengan keterangan komponen sebagai berikut :

S: *State* penempatan kendaraan pada geladak kapal.

$V_{terpilih}$: Himpunan seluruh kendaraan yang berhasil dimuat ke dalam kapal pada *state* S.

$ShippingFee(v)$: Tarif pengiriman dari masing-masing kendaraan v yang diangkut.

$ETA(v)$: Perkiraan waktu kedatangan (*Estimated Time of Arrival*) dalam sisa hari untuk kendaraan v.

α: Parameter pembobot penalti ETA, dengan nilai $\alpha = 5$ pada pengujian sistem

2.1.4. Heuristik

Pengurang $\alpha\, X\, ETA$ merupakan heuristik tambahan yang dibuat kelompok karena setiap kendaraan memiliki sisa waktu kedatangan (Estimated Time of Arrival) yang berbeda. Kendaraan dengan nilai ETA lebih kecil menunjukkan urgensi pengiriman yang lebih tinggi karena tenggat waktu kedatangannya lebih mendesak. Dengan menjadikannya faktor pengurang, algoritma diarahkan untuk memprioritaskan pemuatan kendaraan-kendaraan mendesak ketika menghadapi pilihan beberapa kandidat muatan

2.1.5. Algoritma

Berikut adalah fungsi yang digunakan untuk menghitung objective function :

```
def calculate_fitness(posisi, vehicles, alpha=5):
    total_fee = 0
    total_eta = 0

    for v in vehicles:
        if posisi[v.id] is not None:
            total_fee += v.shipping_fee
            total_eta += v.eta

    return total_fee - (alpha * total_eta)
```

2.2. Implementasi Algoritma Local Search

Berikut merupakan penjelasan implementasi dari setiap jenis *local search* yang digunakan dalam menyelesaikan persoalan.

2.2.1. Simple Hill-Climbing

Simple hill-climbing adalah algoritma *local search* yang pada setiap iterasi memeriksa *neighbor* dari *current state*, lalu langsung berpindah ke *neighbor* pertama dengan nilai yang lebih baik. Jika nilai *neighbor* lebih baik dari *current value*, maka *neighbor* tersebut langsung menjadi *current state*. Jika tidak ada *neighbor* yang lebih baik dalam batas iterasi tertentu (dalam kode kami 1000 iterasi), pencarian berhenti dan *current state* dikembalikan sebagai solusi akhir. Algoritma ini cepat dalam mengambil keputusan, tetapi dapat terjebak di *local optimum*, *ridge*, atau *shoulder*.

Nama File : algorithms.py

Variabel	
current_state	State saat ini
current_fitness	Nilai objective function dari current_state
improved	Flag boolean penanda apakah neighbor yang dicek lebih baik atau tidak.
iteration	Banyak iterasi yang sudah dilakukan
history	List nilai objective function tiap iterasi, untuk plot
max_iter	Batas iterasi
Fungsi dan Prosedur	
simple_hill_climbing(initial_state, ships, vehicles, max_iter=1000)	Fungsi utama: menjalankan algoritma simple_hill_climbing dengan mencari solusi dengan memilih tetangga pertama yang lebih baik. Mengembalikan state akhir, nilai akhir, jumlah iterasi, dan *history*.
get_deterministic_neighbors(posisi, ships, vehicles)	Generator seluruh neighbor valid, dengan urutan rotate, swap, lalu relocate
calculate_fitness(posisi, vehicle, alpha =5)	Fungsi untuk menghitung nilai fungsi objektif dari suatu state

Berikut merupakan potongan source code dari fungsi utamanya

```
def simple_hill_climbing(initial_state, ships, vehicles,
max_iter=1000):
    current_state = {vid: dict(p) if p is not None else
None for vid, p in initial_state.items()}
    current_fitness = calculate_fitness(current_state,
vehicles)

    iteration = 0
    history = [current_fitness]
    print(f"Iterasi {iteration}: Fitness Score =
{current_fitness}")

    while iteration < max_iter:
        iteration += 1
        improved = False

        # Telusuri tetangga terstruktur, langsung lompat
saat menemukan tetangga pertama yang lebih baik (cut)
        for neighbor in
get_deterministic_neighbors(current_state, ships,
vehicles):
            f = calculate_fitness(neighbor, vehicles)
            if f > current_fitness:
                current_state = neighbor
                current_fitness = f
                improved = True
                break

        history.append(current_fitness)

        if improved:
            print(f"Iterasi {iteration}: Fitness Score =
{current_fitness}")
        else:
            print(f"Iterasi {iteration}: Mencapai Local
Optimum (berhenti).")
            break

    return current_state, current_fitness, iteration,
history
```

2.2.2. Hill-Climbing Steepest Ascent

Steepest ascent hill-climbing adalah algoritma *local search* yang pada setiap iterasi memeriksa seluruh *neighbor* dari *current state*, lalu berpindah ke *neighbor* dengan nilai terbaik. Jika nilai *neighbor* terbaik

lebih besar dari *current value*, *neighbor* tersebut menjadi *current state*. Jika tidak ada *neighbor* yang lebih baik, pencarian berhenti dan *current state* dikembalikan sebagai solusi akhir. Algoritma ini cepat dan sederhana, tetapi dapat terjebak di *local optimum* atau *shoulder*.

Nama File : algorithms.py

Variabel	
current_state	State saat ini
current_fitness	Nilai objective function dari current_state
best_neighbor	Neighbor dengan nilai terbaik pada iterasi tersebut (None bila tidak ada yang lebih baik)
best_fitness	Nilai neighbor terbaik pada iterasi tersebut
iteration	Banyak iterasi yang sudah dilakukan
history	List nilai objective function tiap iterasi, untuk plot
max_iter, verbose	Batas iterasi dan pilihan mencetak proses (parameter)
Fungsi dan Prosedur	
steepest_ascent_hill_climbing(initial_state, ships, vehicles, max_iter=1000, verbose=True)	Fungsi utama; mengembalikan state akhir, nilai akhir, jumlah iterasi, dan history
get_deterministic_neighbors(posisi, ships, vehicles)	Generator seluruh neighbor valid, dengan urutan rotate, swap, lalu relocate

Berikut merupakan potongan source code dari fungsi utamanya

```
def steepest_ascent_hill_climbing(initial_state, ships,
vehicles, max_iter=1000, verbose=True):
    current_state = {vid: dict(p) if p is not None else None
for vid, p in initial_state.items()}
    current_fitness = calculate_fitness(current_state,
```

```
vehicles)

    iteration = 0
    history = [current_fitness]
    if verbose:
        print(f"Iterasi {iteration}: Fitness Score =
{current_fitness}")

    while iteration < max_iter:
        iteration += 1
        best_neighbor = None
        best_fitness = current_fitness

        # Evaluasi SEMUA neighbor
        for neighbor in
get_deterministic_neighbors(current_state, ships, vehicles):
            f = calculate_fitness(neighbor, vehicles)
            if f > best_fitness:
                best_fitness = f
                best_neighbor = neighbor

        if best_neighbor is not None:
            current_state = best_neighbor
            current_fitness = best_fitness
            history.append(current_fitness)
            if verbose:
                print(f"Iterasi {iteration}: Fitness Score =
{current_fitness}")
        else:
            # Tidak ada tetangga yang lebih baik-> Local
Optimum
            history.append(current_fitness)
            if verbose:
                print(f"Iterasi {iteration}: Mencapai Local
Optimum (berhenti).")
            break

    return current_state, current_fitness, iteration, history
```

2.2.3. Stochastic Hill Climbing

Stochastic hill-climbing pada implementasi ini membangkitkan satu *neighbor* secara acak (*relocate*, *rotate*, atau *swap*) pada setiap iterasi. Apabila nilai *neighbor* lebih baik dari *current value*, algoritma berpindah ke *neighbor* tersebut. Jika tidak, *current state* dipertahankan. Pencarian dijalankan sebanyak max_iter iterasi. Pemilihan acak membuat jalur pencarian bervariasi antar percobaan, sehingga hasil akhirnya bisa berbeda meskipun dimulai dari *initial state* yang sama.

Nama File : algorithms.py

Variabel	
current_state, current_fitness	State saat ini dan nilainya
neighbor	Satu neighbor acak yang dibangkitkan pada iterasi tersebut (None bila gagal)
f	Nilai objective function dari neighbor
iteration	Banyak iterasi yang sudah dilakukan
history	List nilai objective function tiap iterasi, untuk plot
max_iter	Batas jumlah iterasi (parameter)
Fungsi dan Prosedur	
stochastic_hill_climbing(initial_state, ships, vehicles, max_iter=1000)	Fungsi utama; mengembalikan state akhir, nilai akhir, jumlah iterasi, dan history
get_random_neighbor(posisi, ships, vehicles)	Membangkitkan satu neighbor valid dengan relocate, rotate, atau swap acak; None bila gagal

Berikut merupakan potongan source code dari fungsi utamanya

```
def stochastic_hill_climbing(initial_state, ships, vehicles,
max_iter=1000):
    current_state = {vid: dict(p) if p is not None else None
for vid, p in initial_state.items()}
    current_fitness = calculate_fitness(current_state,
vehicles)

    iteration = 0
    history = [current_fitness]
    print(f"Iterasi {iteration}: Fitness Score =
{current_fitness}")

    for _ in range(max_iter):
        iteration += 1
        neighbor = get_random_neighbor(current_state, ships,
vehicles)

        if neighbor is not None:
            f = calculate_fitness(neighbor, vehicles)
            if f > current_fitness:
                current_state = neighbor
```

```
                current_fitness = f
                print(f"Iterasi {iteration}: Fitness Score =
{current_fitness}")

        history.append(current_fitness)

    return current_state, current_fitness, iteration, history
```

2.2.4. Hill-Climbing with Sideways Move

Hill-climbing with sideways move adalah pengembangan *steepest ascent hill-climbing* yang tetap dapat bergerak ketika *neighbor* terbaik bernilai sama dengan *current value*. Pada *steepest ascent* biasa, pencarian berhenti di area datar karena tidak ada *neighbor* yang lebih baik. Dengan *sideways move*, pencarian dapat melintasi area datar dan berpeluang menemukan jalan naik, dengan syarat jumlah *sideways move* berturut-turut dibatasi oleh parameter *maximum sideways move*.

Pencarian dimulai dari *initial state* acak yang dinilai dengan *objective function*. Pada setiap iterasi, algoritma membangkitkan seluruh *neighbor* (*swap*, *relocate*, dan *rotate*) lalu menghitung nilainya. Bila *neighbor* terbaik lebih baik dari *current value*, algoritma berpindah ke sana dan hitungan *sideways* berturut-turut di-reset menjadi nol. Bila nilainya sama, algoritma berpindah (*sideways move*) selama hitungan berturut-turut belum mencapai batas. Bila neighbor terbaik lebih buruk, atau batas *sideways* tercapai, pencarian berhenti. Jika terdapat beberapa neighbor terbaik dengan nilai sama, satu di antaranya dipilih secara acak. Alasan berhenti dicatat sebagai *local_optimum*, *max_sideways_reached*, atau *max_iterations* (batas iterasi).

Nama File : *hill_climbing_sideways_move.py*

Variabel	
current	*State* saat ini
current_fitness	Nilai *objective function* dari *current*
history	List nilai *objective function* tiap iterasi, untuk *plot*
iteration	Banyak iterasi yang sudah dilakukan

sideways_count	Banyak *sideways move* berturut-turut saat ini; di-*reset* saat ada perbaikan
total_sideways	Total *sideways move* selama pencarian
max_sideways	Batas *sideways move* berturut-turut (parameter)
max_iter	Batas jumlah iterasi (parameter)
neighbors	Daftar seluruh *neighbor* dari *current*
scored_neighbors	Pasangan (nilai, *neighbor*) untuk semua *neighbor*
best_fitness	Nilai *neighbor* terbaik pada iterasi tersebut
best_candidates	Daftar neighbor yang nilainya sama dengan *best_fitness*
stop_reason	Alasan pencarian berhenti
Fungsi dan Prosedur	
hill_climbing_sideways_move(initial_state, ships, vehicles, max_sideways=100, max_iter=10000)	Fungsi utama pencarian; mengembalikan state akhir, nilai akhir, jumlah iterasi, history, total sideways, dan alasan berhenti
get_all_neighbors(posisi, ships, vehicles)	Membangkitkan seluruh neighbor valid dengan swap, relocate, dan rotate
_copy_state(posisi)	Membuat salinan state agar state asli tidak berubah
_state_key(posisi)	Mengubah state menjadi tuple untuk mendeteksi state yang sama

Berikut merupakan potongan *source code* dari fungsi utamanya

```
def hill_climbing_sideways_move(initial_state, ships, vehicles,
max_sideways=100, max_iter=10000):
    current = _copy_state(initial_state)
    current_fitness = calculate_fitness(current, vehicles)
    history = [current_fitness]

    iteration       = 0
```

```
    sideways_count  = 0
    total_sideways  = 0
    stop_reason     = "max_iterations"

    while iteration < max_iter:
        neighbors = get_all_neighbors(current, ships, vehicles)
        if not neighbors:
            stop_reason = "local_optimum"
            break

        scored_neighbors = [(calculate_fitness(n, vehicles), n)
for n in neighbors]
        best_fitness     = max(f for f, _ in scored_neighbors)
        best_candidates  = [n for f, n in scored_neighbors if f
== best_fitness]

        if best_fitness < current_fitness:
            stop_reason = "local_optimum"
            break

        if best_fitness == current_fitness:
            if sideways_count >= max_sideways:
                stop_reason = "local_optimum" if max_sideways
== 0 else "max_sideways_reached"
                break
            sideways_count += 1
            total_sideways += 1
        else :
            sideways_count = 0

        current         = random.choice(best_candidates)
        current_fitness = best_fitness
        iteration      += 1
        history.append(current_fitness)

    return current, current_fitness, iteration, history,
total_sideways, stop_reason
```

2.2.5. Random Restart Hill Climbing

Random restart hill-climbing menjalankan *steepest ascent hill-climbing* berulang kali. *Restart* pertama dimulai dari *initial state* yang diberikan, dan tiap *restart* berikutnya dimulai dari *state* acak baru hasil randomize_state. Dari seluruh *restart*, *state* dengan nilai tertinggi disimpan sebagai solusi akhir. Strategi ini mengurangi risiko terjebak di satu *local optimum*, karena tiap *restart* dapat berakhir di puncak yang berbeda. Pencarian berhenti setelah jumlah *restart* mencapai batas max_restarts.

Nama File : algorithms.py

Variabel	
best_state, best_fitness	Solusi terbaik dari seluruh restart
current_init	State awal untuk restart yang sedang berjalan
local_state, local_fitness	Hasil steepest ascent pada restart tersebut
total_iterations	Total iterasi dari seluruh restart
all_history, best_so_far_history	Nilai objective function tiap iterasi dan nilai terbaik sejauh ini, untuk plot
max_restarts, max_iter	Batas banyak restart dan iterasi tiap restart (parameter)
Fungsi dan Prosedur	
random_restart_hill_climbing(initial_state, ships, vehicles, max_restarts=30, max_iter=1000)	Fungsi utama; mengembalikan state terbaik, nilainya, total iterasi, dan history
steepest_ascent_hill_climbing(...)	Dipanggil pada setiap restart
randomize_state(ships, vehicles)	Membangkitkan state awal acak untuk tiap restart berikutnya

Berikut merupakan potongan source code dari fungsi utamanya

```
def random_restart_hill_climbing(initial_state, ships,
vehicles, max_restarts=30, max_iter=1000):
    best_state = None
    best_fitness = float("-inf")
    total_iterations = 0

    all_history = []
    best_so_far_history = []

    current_init = initial_state

    for i in range(1, max_restarts + 1):
        if i > 1:
            current_init = randomize_state(ships, vehicles)

        local_state, local_fitness, local_iter, local_hist =
steepest_ascent_hill_climbing(
            current_init, ships, vehicles, max_iter=max_iter,
verbose=False
```

```
        )
        total_iterations += local_iter

        # Catat history tiap iterasi untuk plot grafik (dan
running best)
        for h in local_hist:
            all_history.append(h)
            best_candidate = max(best_fitness, h) if
best_fitness != float("-inf") else h
            best_so_far_history.append(best_candidate)

        is_new_best = False
        if local_fitness > best_fitness:
            best_fitness = local_fitness
            best_state = local_state
            is_new_best = True

        status = f" -> Global Best Baru = {best_fitness}" if
is_new_best else ""
        print(f"Restart #{i}/{max_restarts}: Local Optimum =
{local_fitness} (Iterasi: {local_iter}){status}")
```

2.2.6. Simulated Annealing

Simulated annealing adalah algoritma *local search* yang terinspirasi proses pendinginan logam. Pada setiap iterasi, algoritma memilih satu *neighbor* secara acak. Jika *neighbor* lebih baik, algoritma berpindah ke sana. Jika lebih buruk, algoritma tetap dapat berpindah dengan peluang e^(ΔE/T), dengan ΔE selisih nilai *neighbor* terhadap *current value* dan T suhu saat ini. Suhu diturunkan bertahap sehingga pada awal pencarian algoritma lebih sering menerima solusi yang lebih buruk, dan pada akhir pencarian bersikap seperti *hill-climbing*. Mekanisme ini memungkinkan algoritma keluar dari *local optimum*.

Nama File : simulated_annealing.py

Variabel	
initial_state	State awal hasil randomize_state
ships	Daftar objek Ship hasil load_data
vehicles	Daftar objek Vehicle hasil load_data
initial_temp	Suhu awal. Makin tinggi, makin besar peluang menerima solusi yang lebih buruk di awal pencarian

cooling_rate	Faktor pengali penurun suhu tiap iterasi (0 < nilai < 1). Suhu baru = suhu × cooling_rate
stopping_temp	Batas suhu terendah. Pencarian berhenti jika suhu ≤ nilai ini
max_iter	Batas maksimum jumlah iterasi
max_stagnation	Batas iterasi berturut-turut tanpa perbaikan pada fitness terbaik, sebelum pencarian dihentikan
alpha	Bobot ETA pada fungsi fitness (fitness = total fee − alpha × total ETA)
history	Pencatatan jalannya pencarian (berisi kunci pada tabel di bawah). Jika None, tidak ada yang dicatat
current_state	State yang sedang ditempati pencarian
current_fitness	Nilai fitness dari current_state
best_state	State dengan fitness tertinggi yang pernah ditemukan
best_fitness	Fitness dari best_state
temperature	Suhu saat ini, turun tiap iterasi
iterations	Jumlah iterasi yang sudah dijalankan
stagnation_counter	Jumlah iterasi berturut-turut tanpa perbaikan fitness terbaik. Direset ke 0 saat best_fitness membaik
stuck_count	Frekuensi stuck di local optima: iterasi di mana state tidak berpindah, yaitu neighbor lebih buruk yang ditolak atau tidak ada neighbor valid
neighbor	Kandidat state tetangga pada iterasi ini (None jika gagal dibuat)
neighbor_fitness	Fitness dari neighbor

delta	ΔE = neighbor_fitness − current_fitness. Positif berarti lebih baik
probability	Peluang menerima neighbor: 1.0 jika ΔE ≥ 0, selain itu e^(ΔE/T)
accept	Hasil keputusan menerima (True) atau menolak (False) neighbor
Fungsi dan Prosedur	

`simulated_annealing(`
`  initial_state,`
`  ships,`
`  vehicles,`
`  initial_temp=1000.0,`
`  cooling_rate=0.995,`
`  stopping_temp=0.1,`
`  max_iter=10000,`
`  max_stagnation=500,`
`  alpha=5,`
`  history=None`
`)`
	Fungsi utama yang menjalankan Simulated Annealing. Tiap iterasi membuat satu neighbor, menerimanya jika lebih baik atau sama, dan menerima yang lebih buruk dengan peluang e^(ΔE/T), lalu menurunkan suhu. Berhenti jika suhu ≤ stopping_temp, iterasi mencapai max_iter, atau stagnasi mencapai max_stagnation. Mengembalikan (best_state, best_fitness, iterations)

`record_objective()`	Fungsi bersarang. Mencatat fitness saat ini dan fitness terbaik ke history di akhir tiap iterasi
`record(delta, probability, accepted)`	Fungsi bersarang. Mencatat iterasi, suhu, ΔE, peluang, dan status diterima ke history

Berikut merupakan potongan source code dari fungsi utamanya

```
def simulated_annealing(
    initial_state,
    ships,
    vehicles,
    initial_temp=1000.0,
    cooling_rate=0.995,
    stopping_temp=0.1,
    max_iter=10000,
    max_stagnation=500,
    alpha=5,
    history=None
):

    if initial_temp <= 0:
```

```
        raise ValueError("initial_temp harus lebih besar dari
0.")

    if not 0 < cooling_rate < 1:
        raise ValueError("cooling_rate harus di antara 0 dan
1.")

    if not 0 <= stopping_temp < initial_temp:
        raise ValueError(
            "stopping_temp harus >= 0 dan lebih kecil dari
initial_temp."
        )

    if max_iter < 0:
        raise ValueError("max_iter tidak boleh negatif.")

    if max_stagnation < 1:
        raise ValueError("max_stagnation harus minimal 1.")

    current_state = copy_state(initial_state)
    best_state = copy_state(initial_state)

    current_fitness = calculate_fitness(
        current_state, vehicles, alpha
    )
    best_fitness = current_fitness

    temperature = initial_temp
    iterations = 0
    stagnation_counter = 0
    stuck_count = 0

    if history is not None:
        history.clear()
        history.update({
            "iteration": [],
            "temperature": [],
            "delta": [],
            "probability": [],
            "accepted": [],
            "stuck_count": 0,
            "objective": [current_fitness],
            "best_objective": [best_fitness],
        })

    def record_objective():
        if history is not None:
            history["objective"].append(current_fitness)
            history["best_objective"].append(best_fitness)

    def record(delta, probability, accepted):
        if history is not None:
            history["iteration"].append(iterations)
            history["temperature"].append(temperature)
            history["delta"].append(delta)
            history["probability"].append(probability)
```

```
            history["accepted"].append(accepted)

    while (
        temperature > stopping_temp
        and iterations < max_iter
        and stagnation_counter < max_stagnation
    ):
        neighbor = get_random_neighbor(
            current_state, ships, vehicles
        )

        iterations += 1

        if neighbor is None:
            stuck_count += 1
            record(None, None, False)
            record_objective()
            stagnation_counter += 1
            temperature *= cooling_rate
            continue

        neighbor_fitness = calculate_fitness(
            neighbor, vehicles, alpha
        )

        delta = neighbor_fitness - current_fitness

        if delta >= 0:
            probability = 1.0
            accept = True
        else:
            probability = math.exp(delta / temperature)
            accept = random.random() < probability
            if not accept:
                stuck_count += 1

        record(delta, probability, accept)

        if accept:
            current_state = neighbor
            current_fitness = neighbor_fitness

        if current_fitness > best_fitness:
            best_state = copy_state(current_state)
            best_fitness = current_fitness
            stagnation_counter = 0
        else:
            stagnation_counter += 1

        record_objective()
        temperature *= cooling_rate

        if history is not None:
            history["stuck_count"] = stuck_count

    return best_state, best_fitness, iterations
```

2.2.7. Genetic Algorithm

Genetic Algorithm adalah algoritma pencarian berbasis populasi. Pencarian dimulai dari sekumpulan solusi (populasi) yang dibangkitkan secara acak. Setiap solusi dinilai dengan *fitness function*, lalu solusi terbaik dipilih sebagai induk untuk dikombinasikan melalui *crossover* dan dimodifikasi sedikit melalui mutasi sehingga terbentuk generasi berikutnya. Proses ini diulang sebanyak jumlah iterasi (generasi) yang ditentukan.

Pada implementasi ini, satu individu direpresentasikan sebagai state penempatan, yaitu pemetaan setiap kendaraan ke posisi (kapal, koordinat x, y, dan orientasi) atau ke luar kapal. Seleksi induk memakai *tournament selection*, yaitu beberapa individu dipilih acak, lalu yang *fitness*-nya tertinggi menjadi orang tua. *Crossover* dilakukan per kapal dengan garis potong acak: induk pertama mewariskan kendaraan yang berada sebelum garis dan induk kedua mewariskan kendaraan yang berada setelah garis. Kendaraan diletakkan ke anak dengan urutan *ShippingFee* tertinggi lebih dulu dan hanya jika penempatannya *valid* yaitu tidak bertumpuk, di dalam geladak dan tidak melebihi kapasitas sehingga anaknya selalu berupa solusi yang valid. Kendaraan yang belum mendapat tempat akan dicoba untuk ditempatkan di posisi pada kedua induk. Mutasi dilakukan dengan satu langkah *neighbor* acak (*relocate*, *rotate*, atau *swap*) yang sama dengan langkah *neighbor* pada algoritma lain. Individu terbaik pada tiap generasi disalin langsung ke generasi berikutnya (elitisme) sehingga nilai terbaik tidak pernah turun.

Nama File : genetic_algorithm.py

Variabel	
population	Daftar state (individu) pada generasi sekarang
fitnesses	Nilai objective function tiap individu
best_state, best_fitness	Solusi terbaik yang pernah ditemukan
initial_best	Individu terbaik di populasi awal (state awal yang dilaporkan)

history_max, history_avg	Nilai maksimum dan rata-rata populasi tiap generasi, untuk plot
population_size	Jumlah individu dalam populasi (parameter, bawaan 30)
generations	Banyak generasi (parameter, bawaan 100)
crossover_rate	Peluang dua induk digabung (parameter, bawaan 0,9)
mutation_rate	Peluang anak dimutasi (parameter, bawaan 0,5)
tournament_size	Banyak peserta pada tiap turnamen seleksi (parameter, bawaan 3)
elite_size	Jumlah individu terbaik yang langsung lolos (parameter, bawaan 1)
Fungsi dan Prosedur	

`genetic_algorithm(`
`  ships,`
`  vehicles,`
`  population_size=30,`
`  generations=100,`
`  crossover_rate=0.9,`
`  mutation_rate=0.5,`
`  tournament_size=3,`
`  elite_size=1,`
`  seed=None,`
`  verbose=False)`
	Fungsi utama; mengembalikan state terbaik, nilainya, jumlah generasi, history_max, history_avg, dan initial_best

`select_parent(`
`  population,`
`  fitnesses,`
`  tournament_size)`
	Seleksi turnamen: memilih induk dari beberapa individu acak

`crossover(`
`  parent_a,`
`  parent_b,`
`  ships,`
`  vehicles)`
	Menggabungkan dua induk menjadi satu anak yang valid

`pick_by_cut_line(`
`  first,`
`  second,`
`  ships,`
`  vehicles)`
	Langkah 1 crossover: memotong tiap kapal dan mewarisi kendaraan di tiap sisi garis

`place_inherited(`
`  inherited,`
`  ship_by_id,`
`  vehicles)`
	Langkah 2: menempatkan kendaraan warisan ke anak, tarif terbesar lebih dulu

`fill_unplaced(`
`  child,`
`  parent_a,`
`  parent_b,`
`  ship_by_id,`
`  vehicles)`
	Langkah 3: menempatkan kendaraan yang belum dapat tempat

`mutate(`
`  child,`
`  ships,`
`  vehicles)`
	Satu gerakan neighbor acak

`_copy_state(state)`	Membuat salinan state agar state asli tidak berubah

Berikut merupakan potongan source code dari fungsi utamanya

```
def genetic_algorithm(ships, vehicles, population_size=30,
generations=100, crossover_rate=0.9,
                      mutation_rate=0.5, tournament_size=3,
elite_size=1, seed=None, verbose=False):
    if seed is not None:
        random.seed(seed)                                  # jika
seed sama maka hasil bisa diulang

    # populasi awal: banyak state acak
    population = [randomize_state(ships, vehicles) for _ in
range(population_size)]
    fitnesses  = [calculate_fitness(individual, vehicles) for
individual in population]

    best_idx      = fitnesses.index(max(fitnesses))
    initial_best  = _copy_state(population[best_idx])       #
state awal terbaik (untuk ditampilkan)
    best_state    = _copy_state(population[best_idx])       #
solusi terbaik sejauh ini
    best_fitness  = fitnesses[best_idx]

    history_max   = [max(fitnesses)]                        #
nilai terbaik di setiap generasi
    history_avg   = [sum(fitnesses) / len(fitnesses)]       #
rata-rata nilai populasi setiap generasi

    for generation in range(1, generations + 1):
        # individu terbaik langsung lolos ke generasi
berikutnya
        ranking        = sorted(range(len(population)),
key=lambda i: fitnesses[i], reverse=True)
```

```
        new_population = [_copy_state(population[i]) for i in
ranking[:elite_size]]

        # sisa tempat diisi anak-anak baru
        while len(new_population) < population_size:
            parent_a = select_parent(population, fitnesses,
tournament_size)
            parent_b = select_parent(population, fitnesses,
tournament_size)
            if random.random() < crossover_rate:
                child = crossover(parent_a, parent_b, ships,
vehicles)
            else:
                child = _copy_state(parent_a)           # tidak
digabung -> anak = salinan induk
            if random.random() < mutation_rate:
                child = mutate(child, ships, vehicles)
            new_population.append(child)

        population = new_population
        fitnesses  = [calculate_fitness(individual, vehicles)
for individual in population]

        best_idx = fitnesses.index(max(fitnesses))
        history_max.append(max(fitnesses))
        history_avg.append(sum(fitnesses) / len(fitnesses))

        if fitnesses[best_idx] > best_fitness:
            best_fitness = fitnesses[best_idx]
            best_state   = _copy_state(population[best_idx])

    return best_state, best_fitness, generations, history_max,
history_avg, initial_best
```

2.3. Hasil dan Analisis

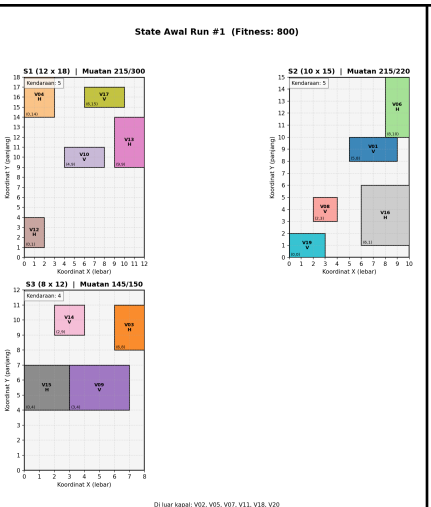
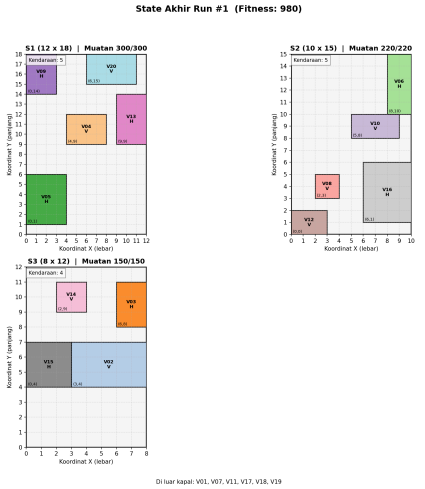
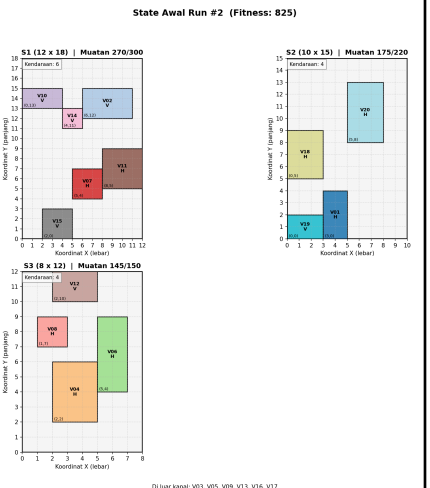
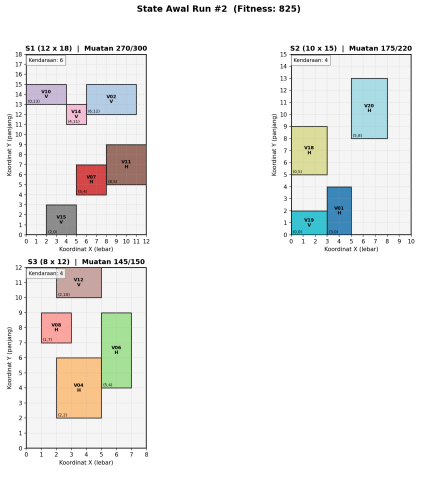
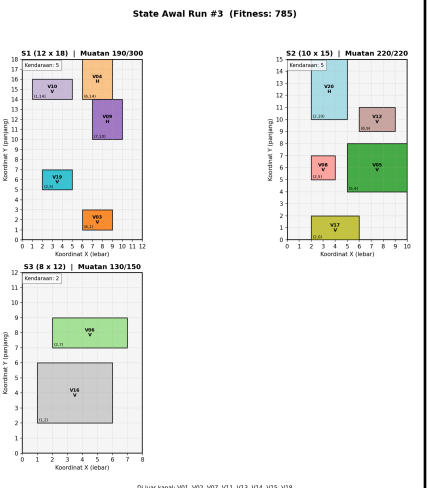
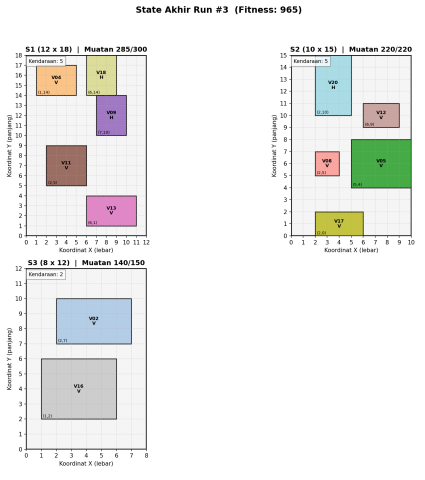
2.3.1. Hasil Eksperimen

2.3.1.1. Simple Hill Climbing

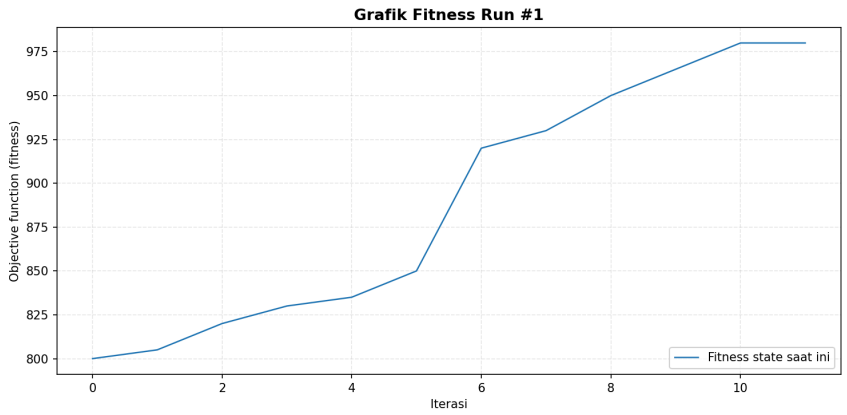
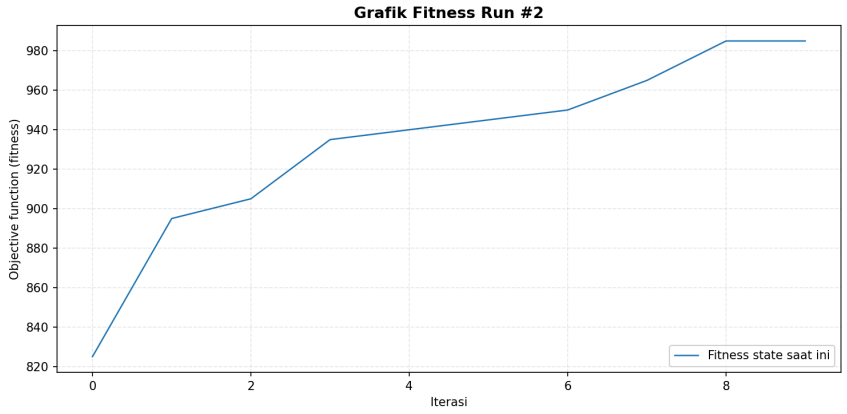
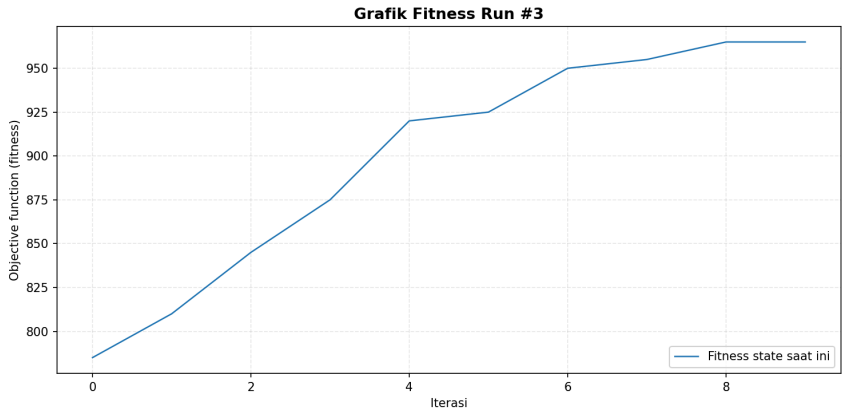
Dilakukan 3 percobaan pada berkas masukan input2.json (3 kapal, 20 kendaraan). Tiap percobaan dimulai dari initial state acak yang berbeda, dengan initial value yang didapat untuk percobaan ini adalah : 800, 825, dan 785.

State awal dan state akhir

Percobaan	State Awal	State Akhir

Percobaan 1		
Percobaan 2		
Percobaan 3		

Plot nilai objective function terhadap banyak iterasi yang telah dilewati

Percobaan	Grafik Objective Function terhadap Iterasi
Percobaan 1	
Percobaan 2	
Percobaan 3	

Ringkasan hasil percobaan

Percobaan	Durasi (s)	Jumlah Iterasi	Initial Value	Final Value
Percobaan 1	0.0232	11	800	980
Percobaan 2	0.0217	9	825	985

Percobaan 3	0.0189	9	785	965

Berdasarkan hasil tiga kali pengujian pada berkas input2.json, algoritma Simple Hill Climbing berhasil meningkatkan nilai evaluasi dari rata-rata initial value sebesar 803,3 (800, 825, dan 785) menjadi rata-rata final value sebesar 976,7 (masing-masing 980, 985, dan 965). Peningkatan ini dicapai dalam jumlah langkah yang sangat singkat, yaitu hanya 11 iterasi pada Percobaan 1 serta 9 iterasi pada Percobaan 2 dan 3. Durasi komputasi yang dibutuhkan juga tergolong cepat, yakni berada pada rentang 0,0189 hingga 0,0232 detik (rata-rata 0,0213 detik).

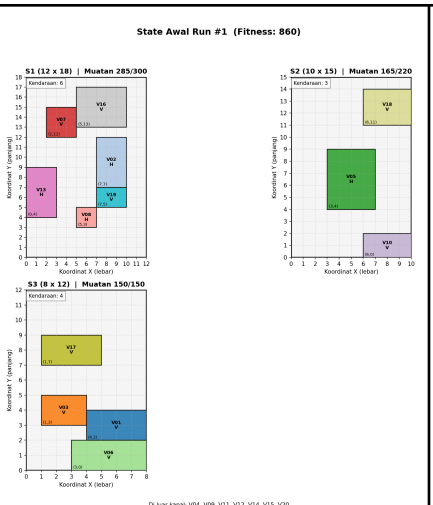
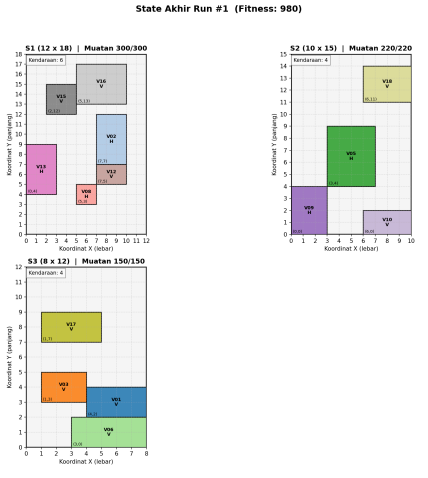
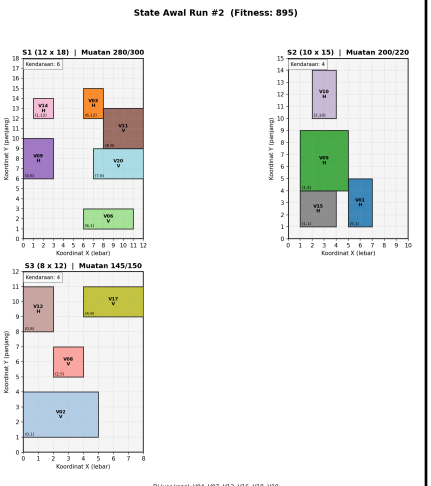
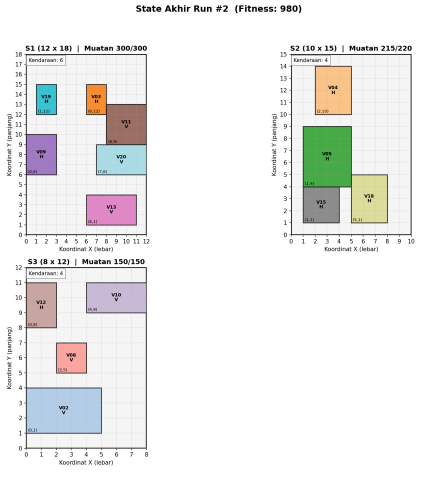
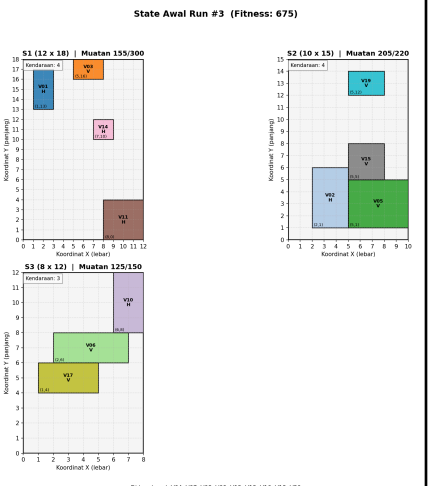
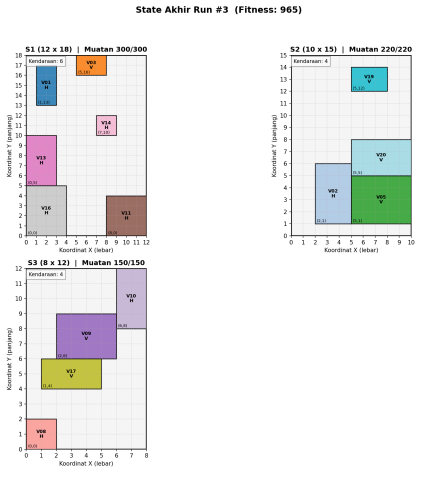
Kecepatan eksekusi dan rendahnya jumlah iterasi tersebut disebabkan oleh mekanisme kerja Simple Hill Climbing yang langsung berpindah begitu menemukan neighbor pertama yang bernilai lebih baik (first-choice), tanpa mengevaluasi ruang tetangga secara menyeluruh. Namun, sifat pemilihan yang tergesa-gesa ini membuat algoritma rentan terjebak di local optimum awal. Begitu tidak ada tetangga searah yang memberikan nilai lebih tinggi, pencarian langsung berhenti. Hasil akhir yang bervariasi (rentang 965–985) membuktikan bahwa kualitas solusi Simple Hill Climbing sangat bergantung pada kualitas dan posisi initial state acak yang dibangkitkan.

2.3.1.2. Hill-Climbing Steepest Ascent

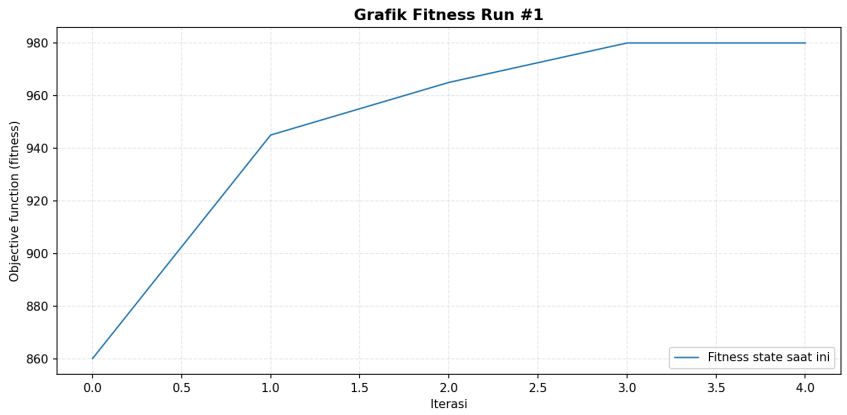
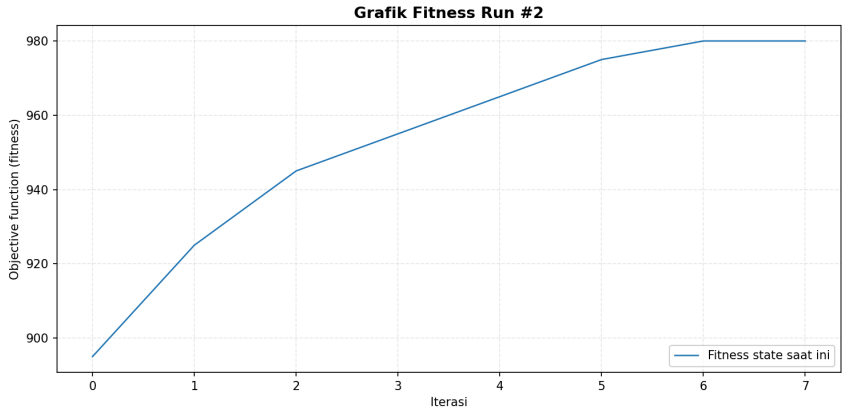
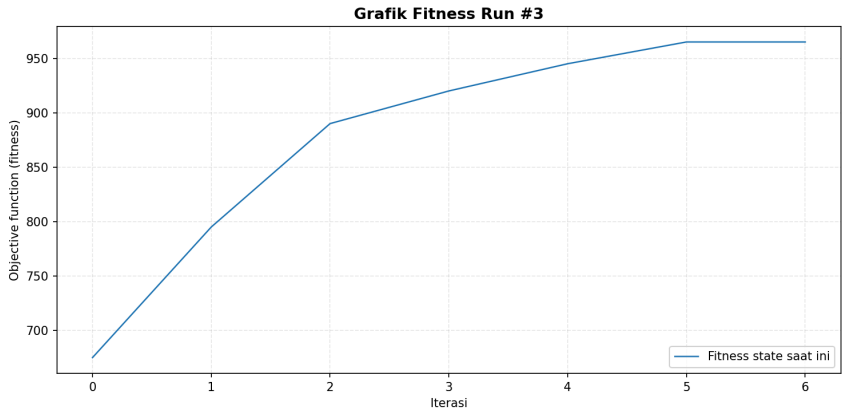
Dilakukan 3 percobaan pada berkas masukan input2.json (3 kapal, 20 kendaraan) dengan parameter max_sideways = 30 dan max_iter = 10000. Tiap percobaan dimulai dari initial state acak yang berbeda, dengan initial value yang didapat adalah : 860, 895, dan 675.

State awal dan state akhir

Percobaan	State Awal	State Akhir

Percobaan 1		
Percobaan 2		
Percobaan 3		

Plot nilai objective function terhadap banyak iterasi yang telah dilewati

Percobaan	Grafik Objective Function terhadap Iterasi
Percobaan 1	
Percobaan 2	
Percobaan 3	

Ringkasan hasil percobaan

Percobaan	Durasi (s)	Jumlah Iterasi	Initial Value	Final Value
Percobaan 1	0.0819	4	860	980
Percobaan 2	0.1441	7	895	980

Percobaan 3	0.1497	6	675	965

Pengujian algoritma Hill-Climbing Steepest Ascent pada tiga percobaan menunjukkan peningkatan nilai dari kondisi awal 860, 895, dan 675 menjadi nilai akhir 980, 980, dan 965 (rata-rata 975,0). Peningkatan terbesar tercatat pada Percobaan 3 dengan kenaikan sebesar 290 poin. Algoritma mencapai titik henti dalam iterasi yang sangat sedikit, yaitu berturut-turut 4, 7, dan 6 iterasi.

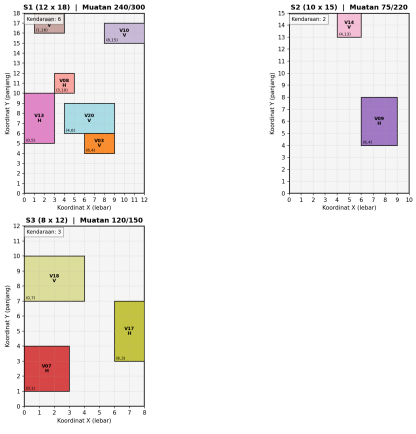
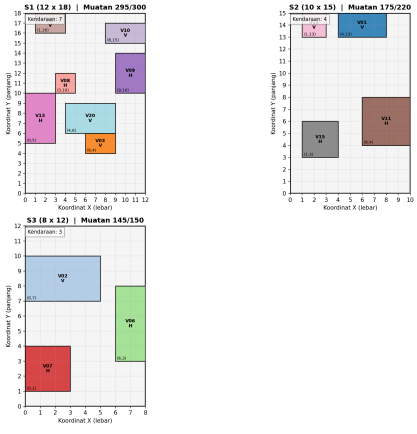
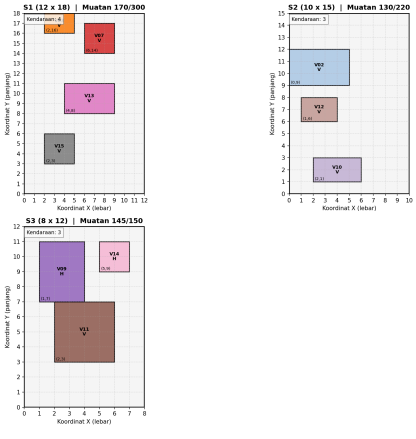
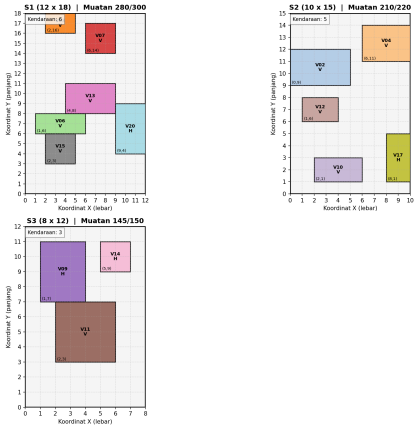
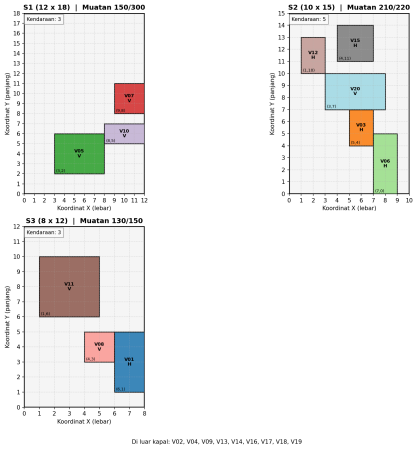
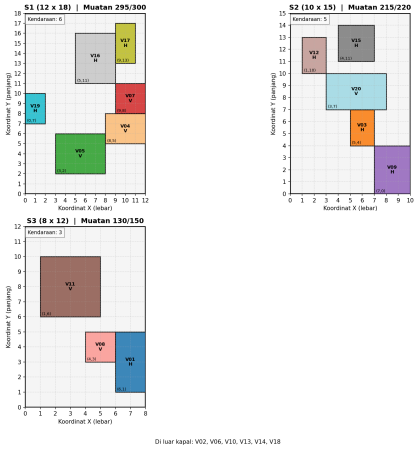
Rendahnya jumlah iterasi ini merupakan karakteristik mendasar Steepest Ascent, di mana pada setiap langkah algoritma mengevaluasi seluruh kemungkinan pergerakan (rotate, swap, relocate) dan memilih kandidat dengan nilai fitness tertinggi (steepest step). Karena mengambil lompatan perbaikan paling maksimal di tiap langkah, kurva konvergensi naik sangat curam dan cepat mencapai puncak lokal. Konsekuensinya, durasi pencarian per iterasi menjadi lebih tinggi dibanding Simple Hill Climbing, berkisar antara 0,0819 hingga 0,1497 detik (rata-rata 0,1252 detik) akibat biaya komputasi pengecekan seluruh neighbor. Seluruh percobaan terhenti murni karena mencapai local optimum kaku tanpa mampu melintasi dataran atau lembah untuk mencari solusi yang lebih mendekati global optimum.

2.3.1.3. Stochastic Hill Climbing

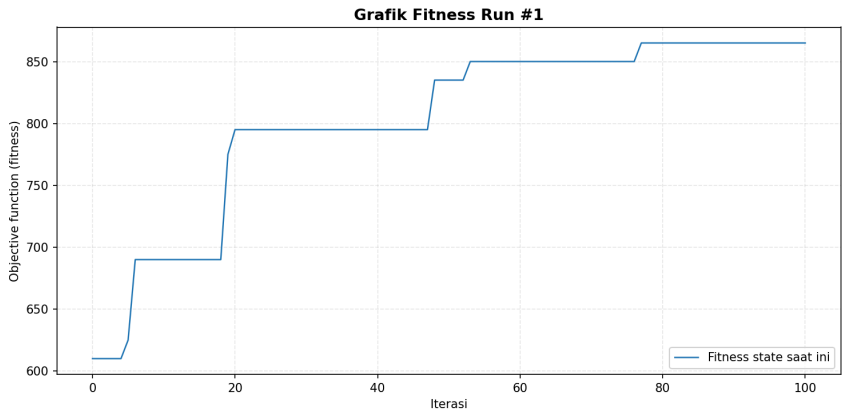
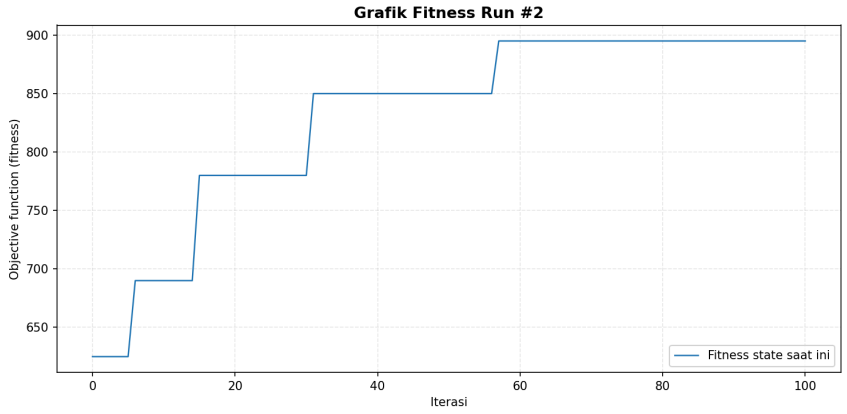
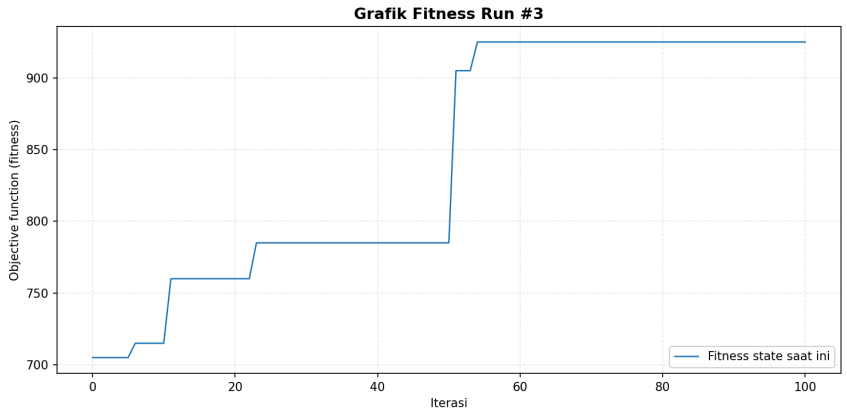
Dilakukan 3 percobaan pada berkas masukan input2.json (3 kapal, 20 kendaraan) dengan parameter max_iter = 100. Tiap percobaan dimulai dari initial state acak yang berbeda, dengan initial value yang didapat untuk percobaan ini adalah : 610, 850, dan 705.

State awal dan state akhir

Percobaan	State Awal	State Akhir

Percobaan 1	State Awal Run #1 (Fitness: 610)  Di luar kapal: V01, V02, V04, V05, V06, V11, V15, V16, V19	State Akhir Run #1 (Fitness: 865)  Di luar kapal: V04, V05, V16, V17, V18, V19
Percobaan 2	State Awal Run #2 (Fitness: 625)  Di luar kapal: V01, V04, V05, V06, V08, V16, V17, V18, V19, V20	State Akhir Run #2 (Fitness: 895)  Di luar kapal: V01, V05, V08, V16, V18, V19
Percobaan 3	State Awal Run #3 (Fitness: 705)  Di luar kapal: V02, V04, V09, V13, V14, V16, V17, V18, V19	State Akhir Run #3 (Fitness: 925)  Di luar kapal: V02, V06, V10, V13, V14, V18

Plot nilai objective function terhadap banyak iterasi yang telah dilewati

Percobaan	Grafik Objective Function terhadap Iterasi
Percobaan 1	
Percobaan 2	
Percobaan 3	

Ringkasan hasil percobaan

Percobaan	Durasi (s)	Jumlah Iterasi	Initial Value	Final Value
Percobaan 1	0.0011	100	610	865
Percobaan 2	0.0013	100	625	895

Percobaan 3	0.0012	100	705	925

Pada pengujian Stochastic Hill Climbing dengan parameter batas 100 iterasi, algoritma mencatatkan perbaikan nilai dari initial value 610, 625, dan 705 menjadi nilai akhir berturut-turut 865, 895, dan 925 (rata-rata 895,0). Dari segi efisiensi waktu, algoritma ini mencatat durasi tercepat dibanding seluruh metode yang diuji, yakni hanya berkisar 0,0011 hingga 0,0013 detik untuk menyelesaikan 100 iterasi. Hal ini terjadi karena algoritma hanya membangkitkan dan mengevaluasi satu neighbor acak per iterasi tanpa adanya pencarian tetangga yang komprehensif.

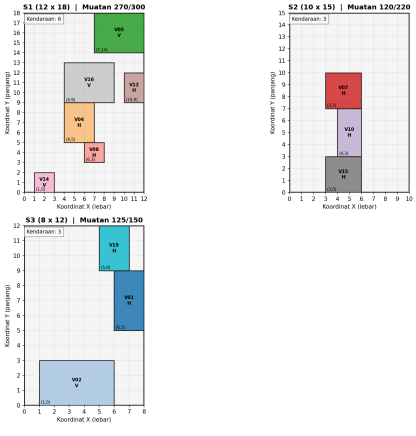
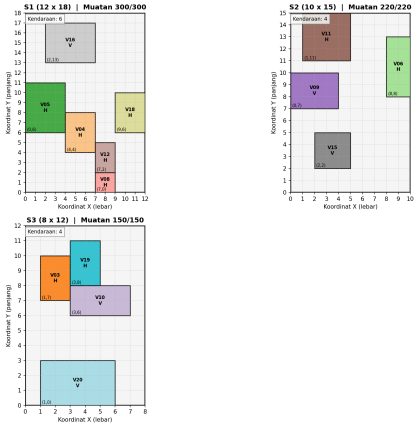
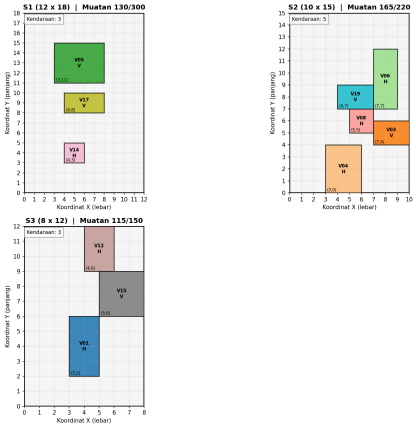
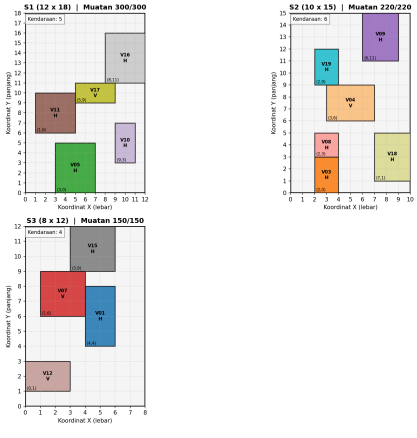
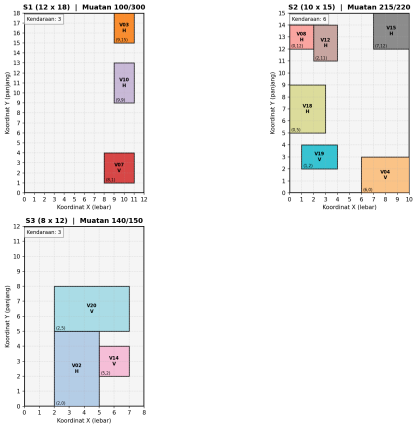
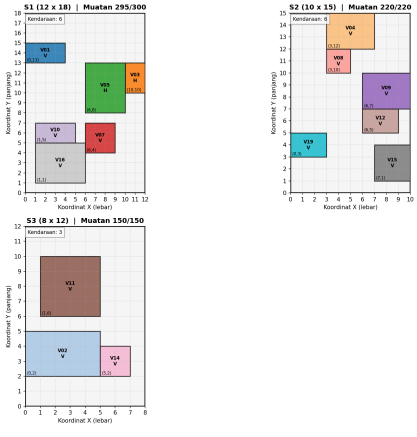
Meskipun sangat ringan secara komputasi, kualitas solusi akhir Stochastic Hill Climbing adalah yang terendah di antara varian hill climbing lainnya. Pemilihan satu kandidat acak menyebabkan banyak iterasi terbuang tanpa adanya perbaikan posisi jika tetangga yang terpilih bernilai lebih rendah atau tidak valid. Akibatnya, dalam alokasi 100 iterasi, algoritma belum sempat mengeksplorasi ruang solusi secara mendalam dan sudah terlanjur berhenti sebelum mencapai kualitas penempatan kendaraan yang optimal.

2.3.1.4. Hill-Climbing with Sideways Move

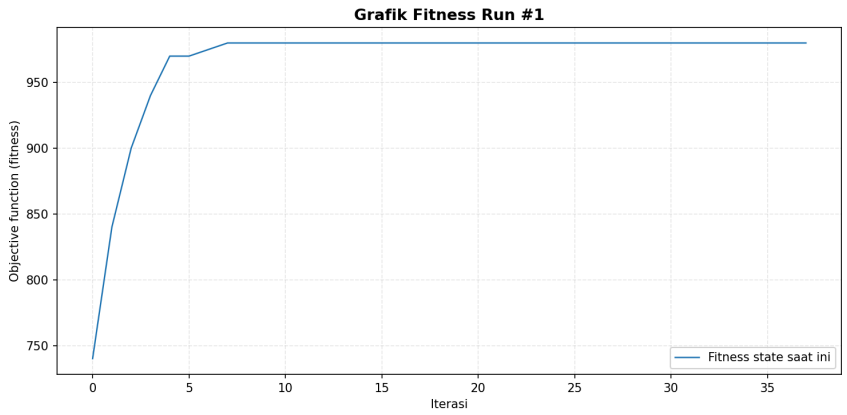
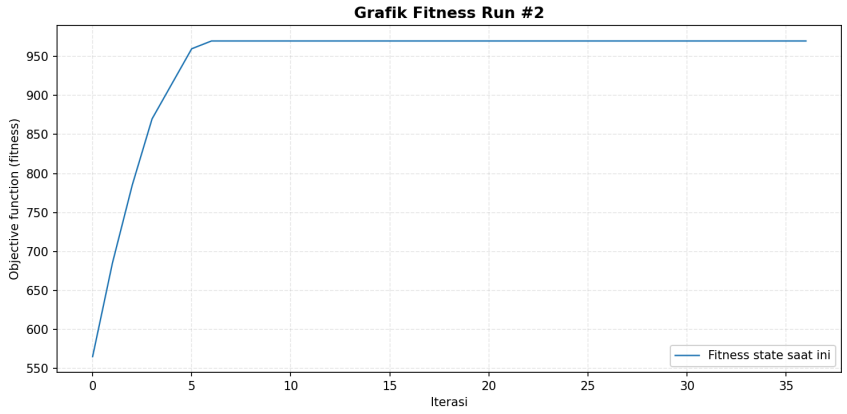
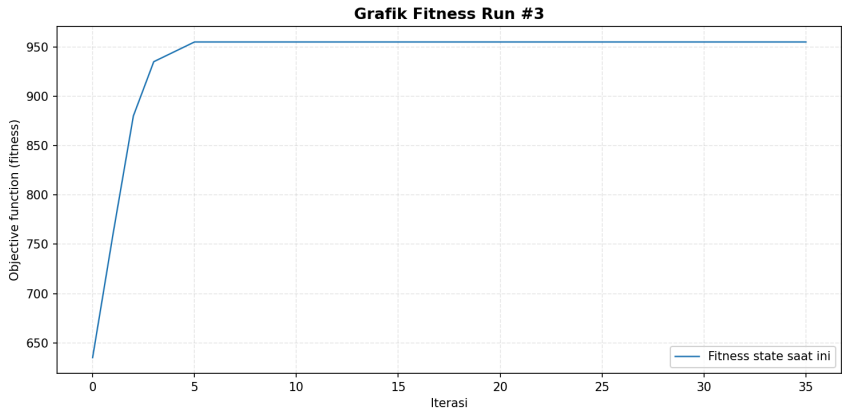
Dilakukan 3 percobaan pada berkas masukan input2.json dengan parameter *max_sideways* = 30 dan *max_iter* = 10000. *Initial state* tiap percobaan dibangkitkan secara acak, dengan nilai awal 740 (Percobaan 1), 565 (Percobaan 2), dan 635 (Percobaan 3).

State awal dan state akhir

Percobaan	**State Awal**	**State Akhir**

Percobaan 1	State Awal Run #1 (Fitness: 740)  Di luar kapal: V03, V06, V09, V11, V13, V17, V18, V20	State Akhir Run #1 (Fitness: 980)  Di luar kapal: V01, V02, V07, V13, V14, V17
Percobaan 2	State Awal Run #2 (Fitness: 565)  Di luar kapal: V02, V07, V09, V10, V11, V13, V16, V18, V20	State Akhir Run #2 (Fitness: 970)  Di luar kapal: V02, V06, V13, V14, V20
Percobaan 3	State Awal Run #3 (Fitness: 635)  Di luar kapal: V01, V05, V06, V09, V11, V13, V16, V17	State Akhir Run #3 (Fitness: 955)  Di luar kapal: V06, V13, V17, V18, V20

Plot nilai objective function terhadap banyak iterasi yang telah dilewati

Percobaan	Grafik Objective Function terhadap Iterasi
Percobaan 1	
Percobaan 2	
Percobaan 3	

Ringkasan hasil percobaan

Percobaan	Durasi (s)	Jumlah Iterasi	Initial Value	Final Value	Max Sideways Move	Total Sideways Move	Penyebab Berhenti

Percobaan 1	1,5828	37	740	980	30	31	max_sideways_reached
Percobaan 2	2,7423	36	565	970	30	30	max_sideways_reached
Percobaan 3	2,7587	35	635	955	30	30	max_sideways_reached

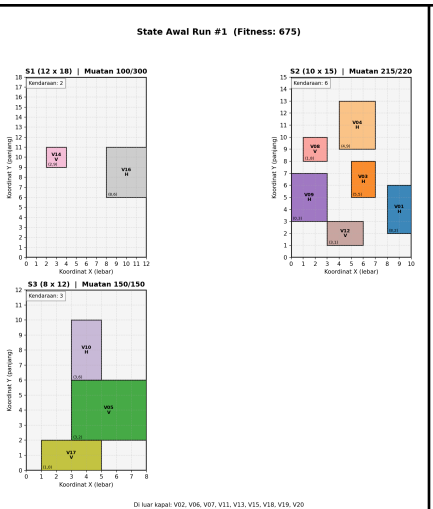
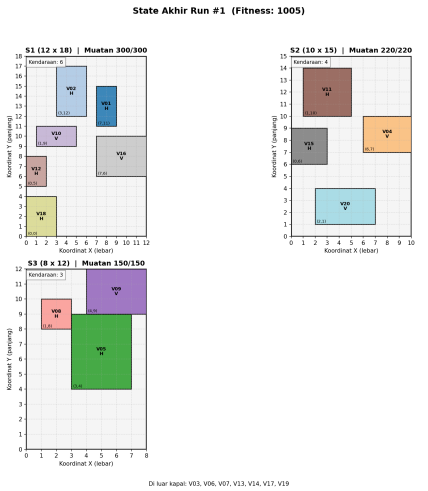
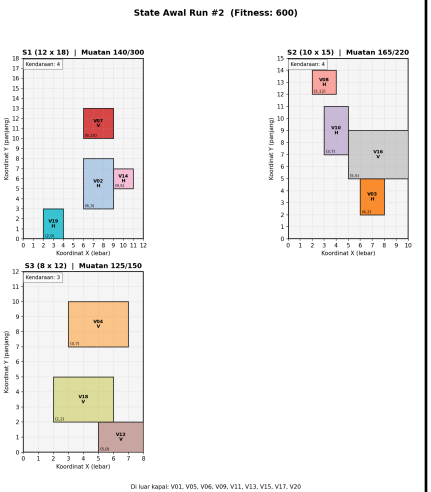
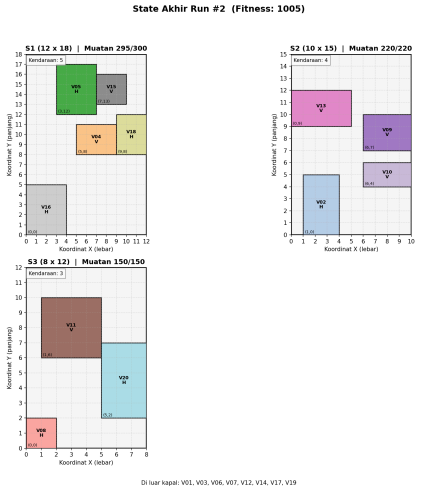
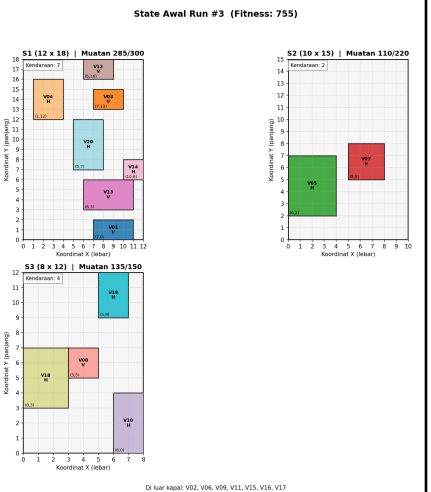
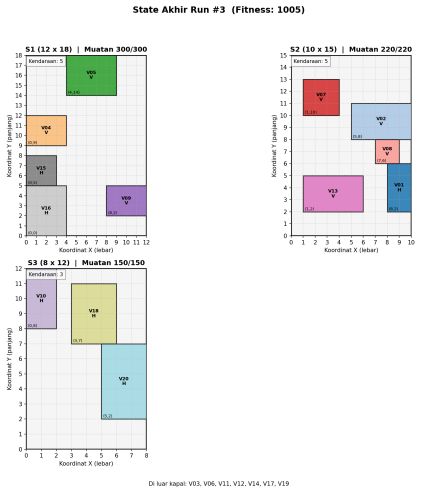
Pada tiga kali pengujian, algoritma Hill-Climbing with Sideways Move berhasil meningkatkan nilai rata-rata dari 646,7 menjadi 968,3 (dengan rincian hasil 980, 970, dan 955), yang mencerminkan kenaikan sebesar 321,7 poin. Peningkatan nilai tersebut terjadi secara signifikan pada 5 hingga 7 iterasi awal sebelum akhirnya konvergen dan cenderung konstan. Seluruh percobaan terhenti akibat tercapainya batas maksimum sideways move (yaitu 30 kali), bukan karena ditemukannya local optimum yang sejati. Kondisi ini mengindikasikan bahwa pencarian terjebak pada area datar (flat local maximum), di mana alokasi 30 kali gerakan menyamping belum memadai untuk menemukan neighbor dengan nilai evaluasi yang lebih tinggi. Keberagaman hasil akhir antar-percobaan (955 hingga 980) dipengaruhi oleh variasi initial state yang dibangkitkan secara acak, yang menunjukkan bahwa algoritma ini memiliki tingkat konsistensi yang terbatas.

2.3.1.5. Random Restart Hill Climbing

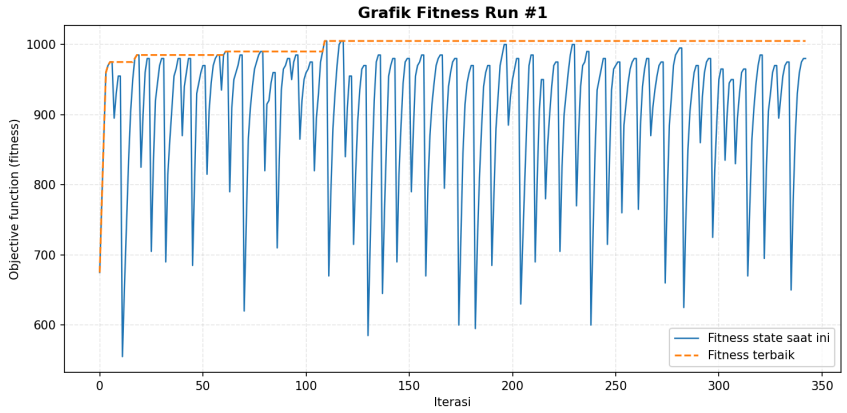
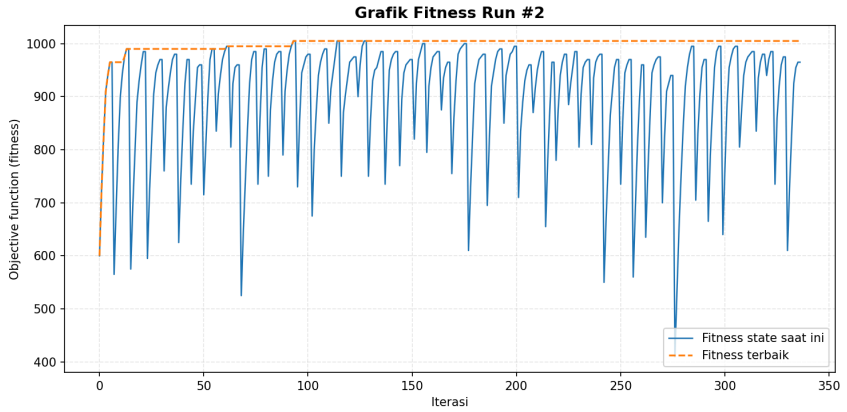
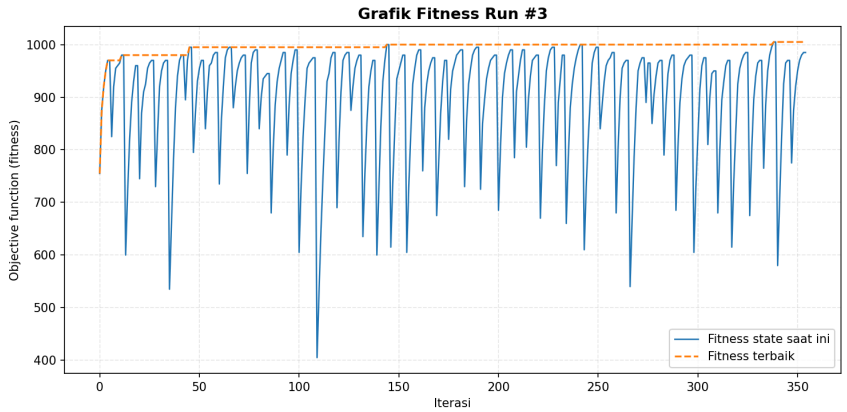
Dilakukan 3 percobaan pada berkas masukaninput2.json (3 kapal, 20 kendaraan) dengan parameter max_restart = 50. Tiap percobaan dimulai dari initial state acak yang berbeda, dengan initial value yang didapat untuk percobaan ini adalah : 675, 600, dan 755.

State awal dan state akhir

Percobaan	State Awal	State Akhir

Percobaan 1		
Percobaan 2		
Percobaan 3		

Plot nilai objective function terhadap banyak iterasi yang telah dilewati

Percobaan	Grafik Objective Function terhadap Iterasi
Percobaan 1	
Percobaan 2	
Percobaan 3	

Ringkasan hasil percobaan

Percobaan	Durasi (s)	Banyak Restart	Total Iterasi	Initial Value	Final Value
Percobaan 1	6.675	50	293	675	1005

Percobaan 2	6.7041	50	287	600	1005
Percobaan 3	6.9979	50	305	755	1005

Hasil eksperimen Random Restart Hill Climbing dengan 50 kali restart menunjukkan performa optimal dan konsistensi tinggi. Dari ketiga percobaan yang dijalankan dengan initial value awal yang bervariasi (675, 600, dan 755), seluruhnya berhasil mencapai nilai akhir yang identik, yaitu 1005. Nilai 1005 ini merupakan global optimum untuk permasalahan pada input2.json, yaitu batas atas teoritis pemilihan muatan berdasarkan kapasitas berat maksimum kapal. Total iterasi kumulatif yang dijalankan berkisar antara 287 hingga 305 iterasi dengan durasi pencarian sekitar 6,675 hingga 6,998 detik.

Grafik pencarian memperlihatkan fluktuasi nilai fitness yang berulang akibat proses re-inisialisasi state secara acak di setiap restart. Meskipun sebagian besar restart individu berakhir pada puncak lokal yang berbeda-beda (sering kali berada di rentang 900–980), strategi diversifikasi pencarian ini berhasil meloloskan algoritma dari jebakan local optimum lokal. Garis best-so-far secara bertahap terangkat hingga akhirnya mengunci nilai optimal global 1005. Trade-off utama dari metode ini adalah durasi komputasi yang jauh lebih lama dibandingkan varian hill climbing tunggal, yang merupakan konsekuensi wajar dari pengulangan proses steepest ascent sebanyak 50 kali.

2.3.1.6. Simulated Annealing

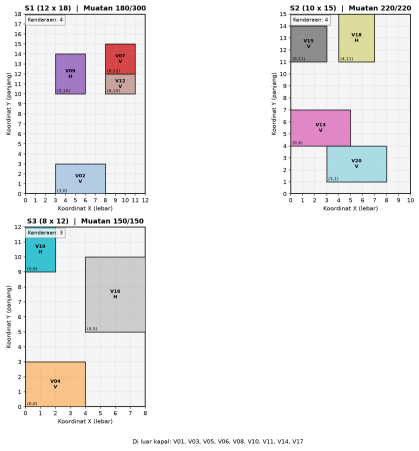
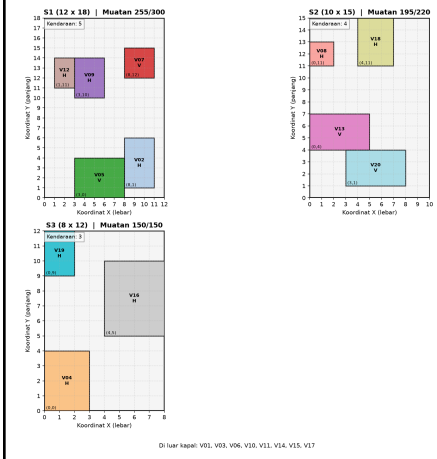
Dilakukan 3 percobaan pada berkas masukan input2.json dengan parameter initial_state, Ships, Vehicles, initial_temp=1000.0, cooling_rate=0.995, stopping_temp=0.1, max_iter=10000, max_stagnation=500, alpha=5, history=None.

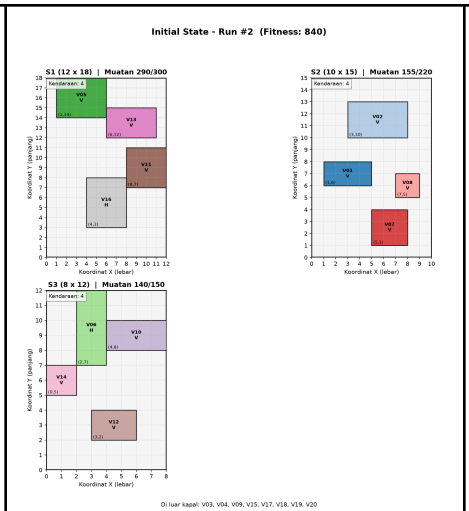
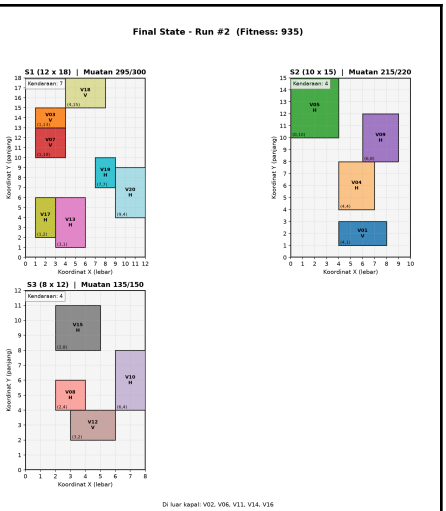
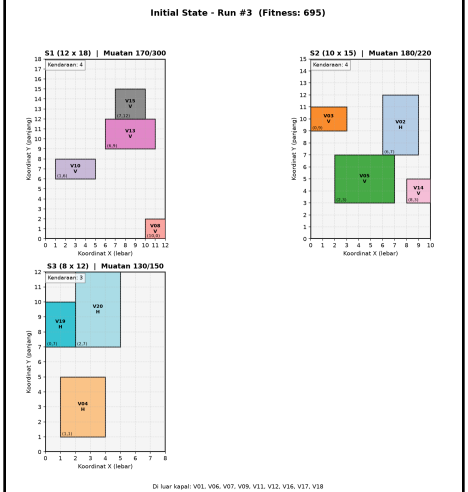
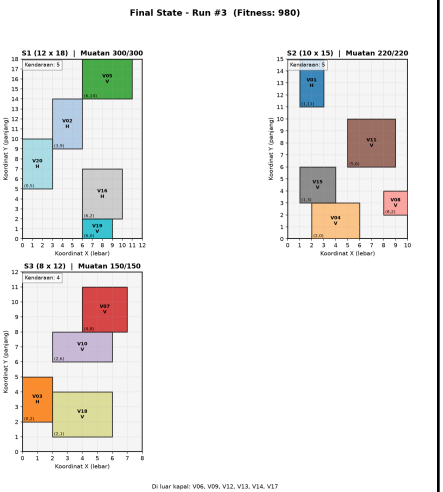
Ship			
ID	**Width (w)**	**Length (l)**	**Max Capacity**
S1	12	18	300
S2	10	15	220
S3	8	12	150

Vehicle						
ID	**Width (w)**	**Length (l)**	**Shipping Fee**	**Weight**	**Orientation**	**ETA**
V01	2	4	65	40	Horizontal	2
V02	3	5	95	60	Horizontal	1
V03	2	3	45	25	Horizontal	3
V04	3	4	80	45	Horizontal	2
V05	4	5	130	75	Horizontal	1
V06	2	5	85	50	Horizontal	4
V07	3	3	60	35	Horizontal	3
V08	2	2	40	20	Horizontal	2
V09	3	4	90	55	Horizontal	1
V10	2	4	70	40	Horizontal	2
V11	4	4	115	70	Horizontal	3
V12	2	3	50	30	Horizontal	2
V13	3	5	100	65	Horizontal	1

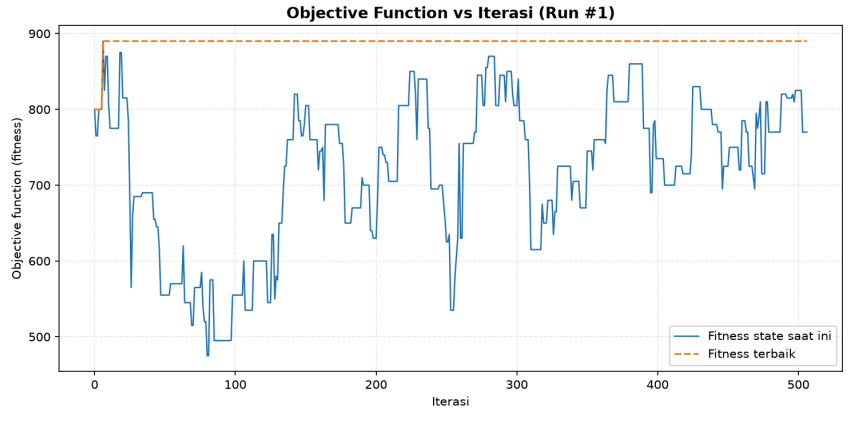
V14	2	2	35	20	Horizontal	4
V15	3	3	75	45	Horizontal	2
V16	4	5	125	80	Horizontal	1
V17	2	4	60	35	Horizontal	3
V18	3	4	85	50	Horizontal	2
V19	2	3	45	25	Horizontal	4
V20	3	5	95	60	Horizontal	1

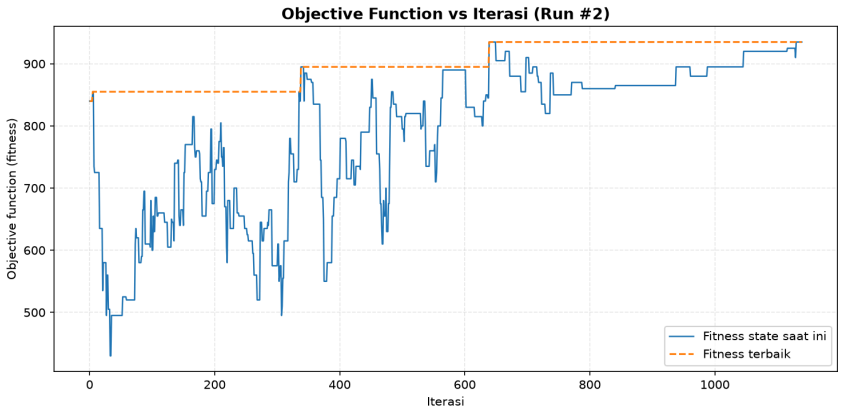
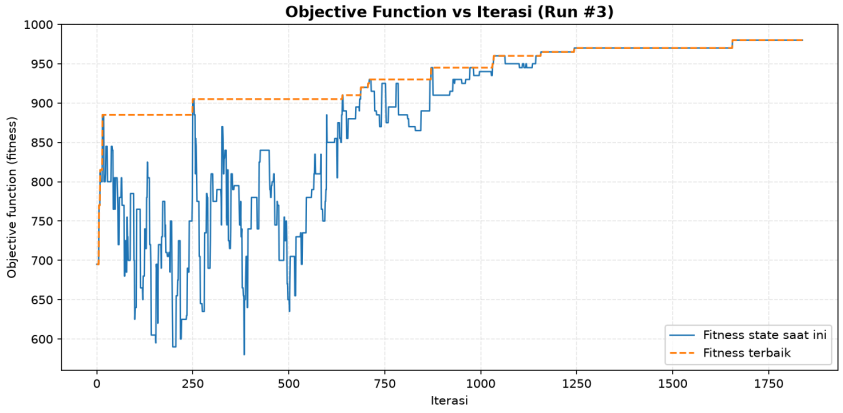
State awal dan state akhir

Percobaan	State Awal	State Akhir
Percobaan 1		

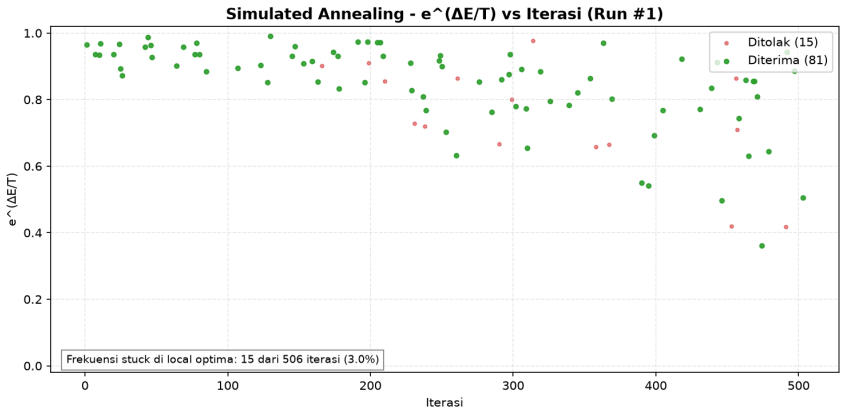
Percobaan 2		
Percobaan 3		

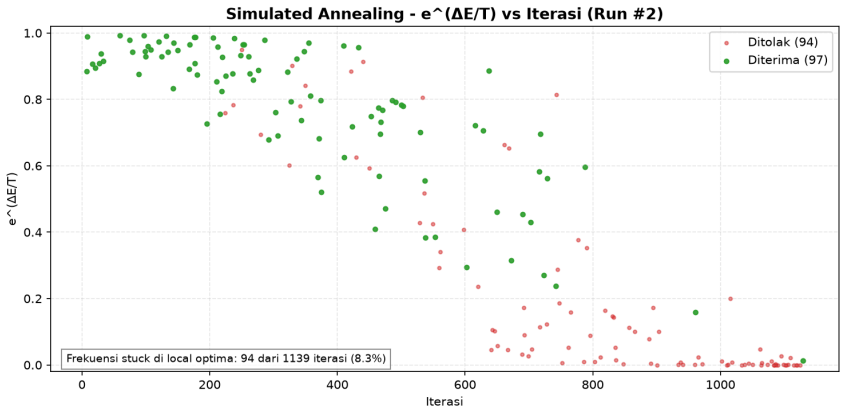
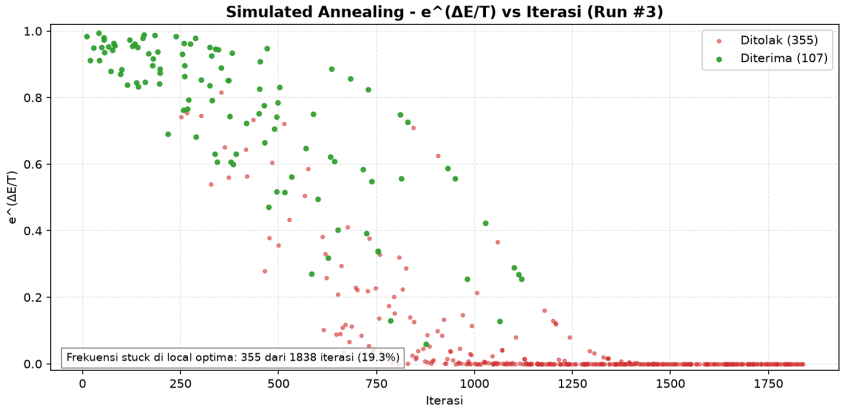
Plot nilai objective function terhadap banyak iterasi yang telah dilewati

Percobaan	Grafik Objective Function terhadap Iterasi
Percobaan 1	

Percobaan 2	
Percobaan 3	

Plot e^(ΔE/T) terhadap banyak iterasi yang telah dilewati

Percobaan	Grafik e^(ΔE/T) terhadap Iterasi
Percobaan 1	

Percobaan 2	
Percobaan 3	

Ringkasan hasil percobaan

Percobaan	Durasi (s)	Jumlah Iterasi	Initial Value	Final Value	Frekuensi Stock di Local Optima
Percobaan 1	0.4174	506	800	890	15 kali dari 506 iterasi
Percobaan 2	0.2060	1139	840	935	94 kali dari 1139 iterasi
Percobaan 3	0.2308	1838	695	980	355 kali dari 1838 iterasi

Berdasarkan hasil tiga percobaan, algoritma *Simulated Annealing* berhasil meningkatkan nilai solusi dari nilai awal 800, 840, dan 695 menjadi 890, 935, dan 980. Peningkatan terbesar terjadi pada Percobaan 3, yaitu sebesar 285 poin. Hal ini menunjukkan bahwa algoritma mampu menemukan solusi yang lebih baik, meskipun belum dapat dipastikan telah mencapai *global optimum*.

Dari segi durasi, Percobaan 1 membutuhkan waktu 0,4174 detik dengan 506 iterasi, Percobaan 2 selama 0,2060 detik dengan 1.139 iterasi, dan Percobaan 3 selama 0,2308 detik dengan 1.838 iterasi. Perbedaan ini menunjukkan bahwa jumlah iterasi tidak selalu berbanding lurus dengan durasi pencarian.

Frekuensi *stuck di local optimum* meningkat dari 15 kali pada Percobaan 1 menjadi 94 kali pada Percobaan 2 dan 355 kali pada Percobaan 3. Hal ini dipengaruhi oleh sifat stokastik algoritma dan jalur pencarian solusi yang berbeda. Namun, tingginya frekuensi *stuck* tidak selalu menunjukkan hasil yang buruk, terbukti Percobaan 3 menghasilkan nilai akhir tertinggi.

Secara keseluruhan, *Simulated Annealing* mampu memperbaiki solusi awal dalam waktu relatif singkat. Namun, diperlukan pengujian tambahan untuk memastikan konsistensi hasil dan kedekatannya dengan *global optimum*.

2.3.1.7. Genetic Algorithm

Genetic Algorithm dijalankan pada input2.json (3 kapal dan 20 kendaraan). Parameter yang diubah-ubah dalam eksperimen adalah jumlah populasi dan banyak iterasi (generasi), sedangkan parameter lain dibuat tetap, yaitu crossover rate 0,9, mutation rate 0,5, tournament size 3, dan elite size 1. Pada Eksperimen 1, jumlah populasi dijadikan kontrol (30) dengan tiga variasi banyak iterasi, yaitu [10, 50, 100]. Pada Eksperimen 2, banyak iterasi dijadikan kontrol (50) dengan tiga variasi jumlah populasi, yaitu [10, 30, 60]. Setiap konfigurasi dijalankan 3 kali. State awal yang ditampilkan adalah individu terbaik pada populasi awal (generasi 0), sehingga Initial Value sama dengan nilai awal pada grafik. (Temabhan : Gambar yang ditampilkan untuk tiap variasi berasal dari Percobaan 1)

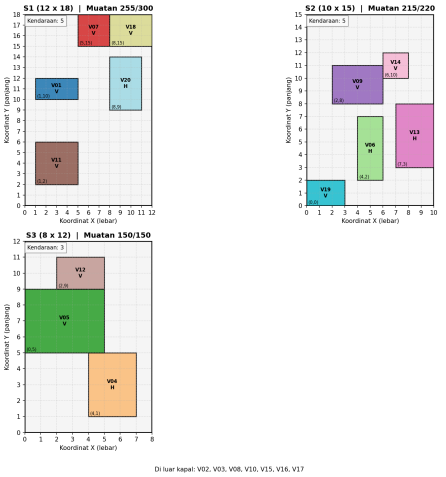
Eksperimen 1: jumlah populasi sebagai kontrol, 3 variasi banyak iterasi, masing-masing dijalankan 3 kali

State awal dan state akhir

Variasi	State Awal	State Akhir

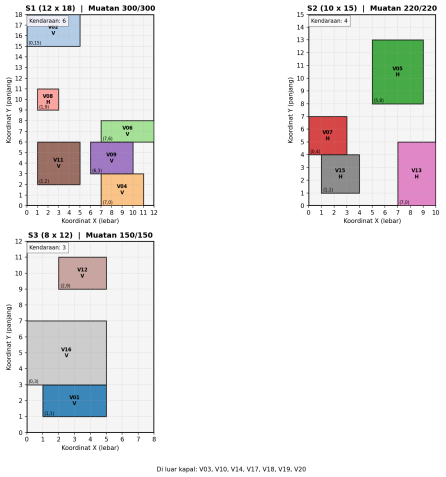
Variasi 1

State Awal (Best Gen 0) Run #1 (Fitness: 885)



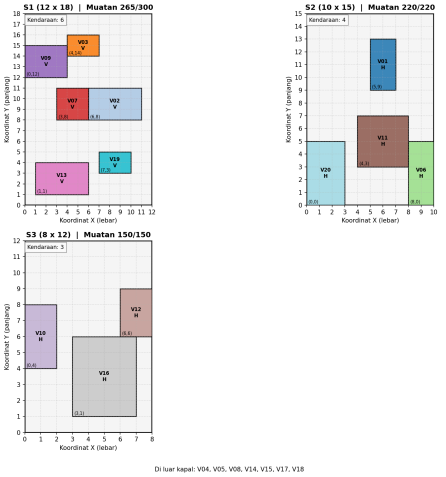
Di luar kapal: V02, V03, V08, V10, V15, V16, V17

State Akhir Run #1 (Fitness: 985)



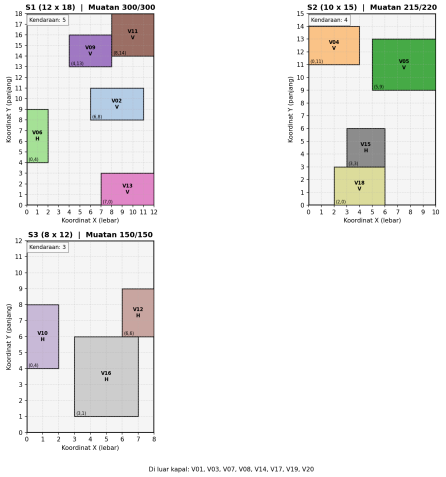
Di luar kapal: V03, V10, V14, V17, V18, V19, V20

State Awal (Best Gen 0) Run #2 (Fitness: 900)



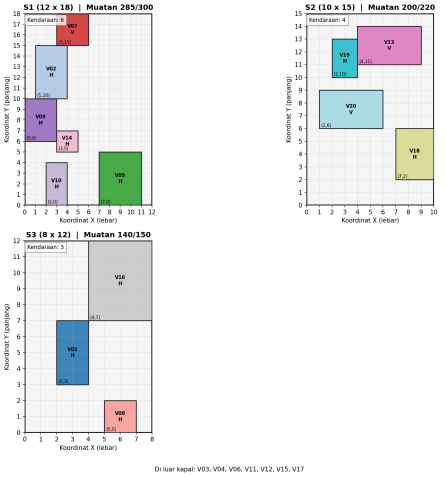
Di luar kapal: V04, V05, V08, V14, V15, V17, V18

State Akhir Run #2 (Fitness: 990)



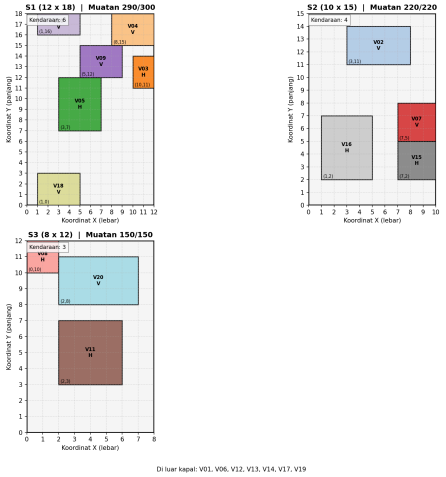
Di luar kapal: V01, V03, V07, V08, V14, V17, V19, V20

State Awal (Best Gen 0) Run #3 (Fitness: 910)



Di luar kapal: V03, V04, V06, V11, V12, V15, V17

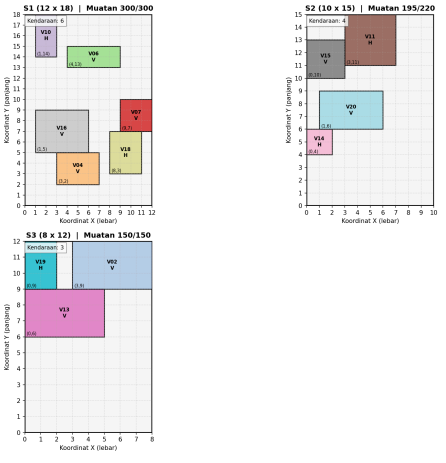
State Akhir Run #3 (Fitness: 985)



Di luar kapal: V01, V06, V12, V13, V14, V17, V19

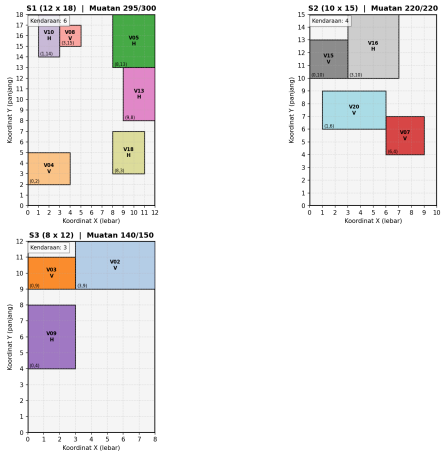
Variasi 2

State Awal (Best Gen 0) Run #1 (Fitness: 915)



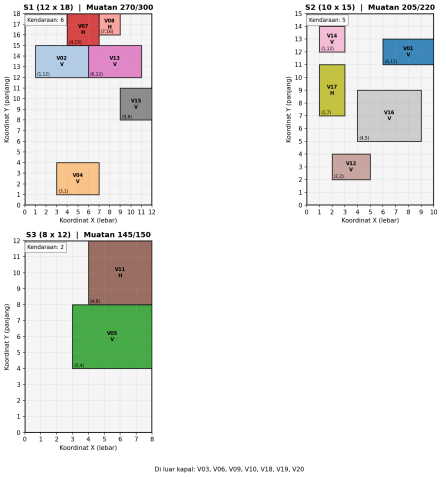
Di luar kapal: V01, V03, V05, V08, V09, V12, V17

State Akhir Run #1 (Fitness: 980)



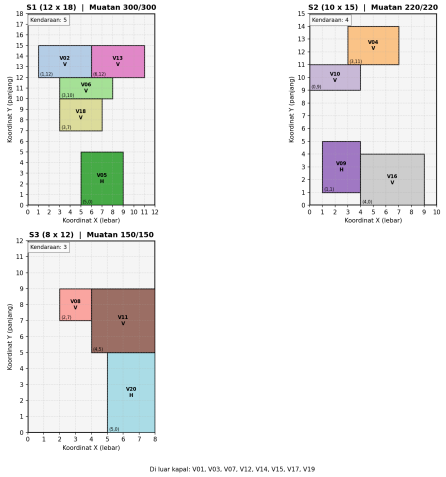
Di luar kapal: V01, V06, V11, V12, V14, V17, V19

State Awal (Best Gen 0) Run #2 (Fitness: 895)



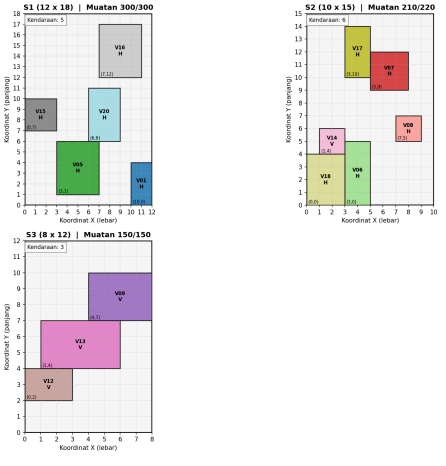
Di luar kapal: V03, V06, V09, V10, V18, V19, V20

State Akhir Run #2 (Fitness: 1005)



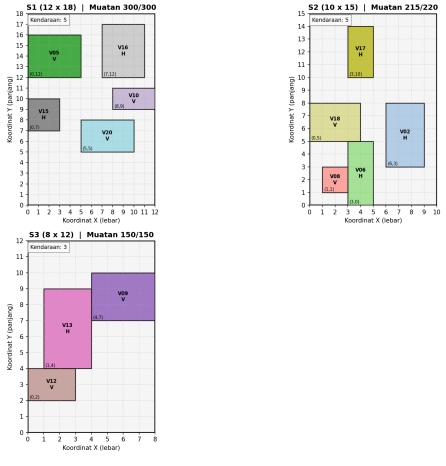
Di luar kapal: V01, V03, V07, V12, V14, V15, V17, V19

State Awal (Best Gen 0) Run #3 (Fitness: 950)



Di luar kapal: V02, V03, V04, V10, V11, V19

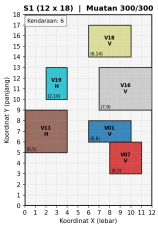
State Akhir Run #3 (Fitness: 985)

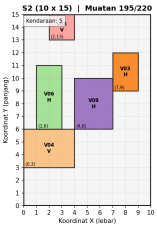


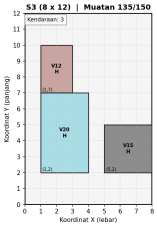
Di luar kapal: V01, V03, V04, V07, V11, V14, V19

Variasi 3

State Awal (Best Gen 0) Run #1 (Fitness: 895)

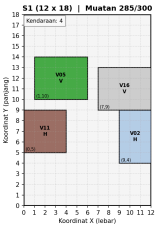


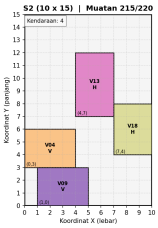


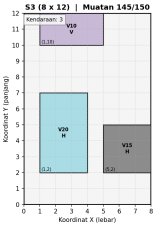


Di luar kapal: V02, V05, V10, V13, V14, V17

State Akhir Run #1 (Fitness: 975)

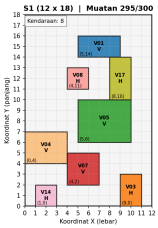


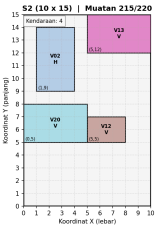


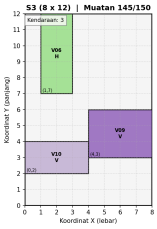


Di luar kapal: V01, V03, V06, V07, V08, V12, V14, V17, V19

State Awal (Best Gen 0) Run #2 (Fitness: 940)

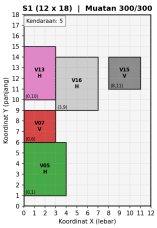


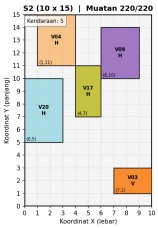


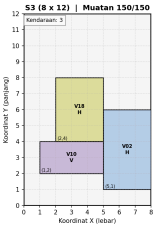


Di luar kapal: V11, V15, V16, V18, V19

State Akhir Run #2 (Fitness: 995)

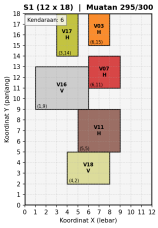


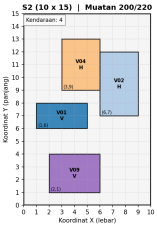


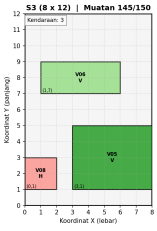


Di luar kapal: V01, V06, V08, V11, V12, V14, V19

State Awal (Best Gen 0) Run #3 (Fitness: 935)

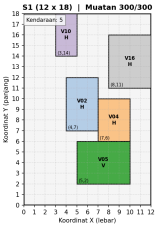


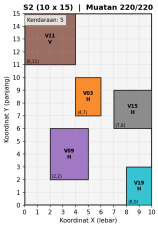


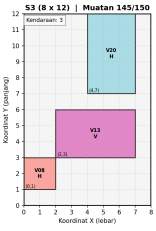


Di luar kapal: V10, V12, V13, V14, V15, V19, V20

State Akhir Run #3 (Fitness: 985)

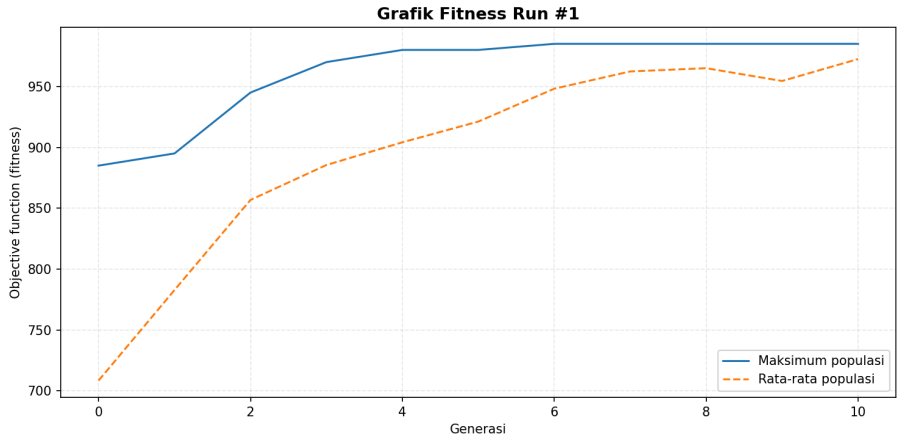
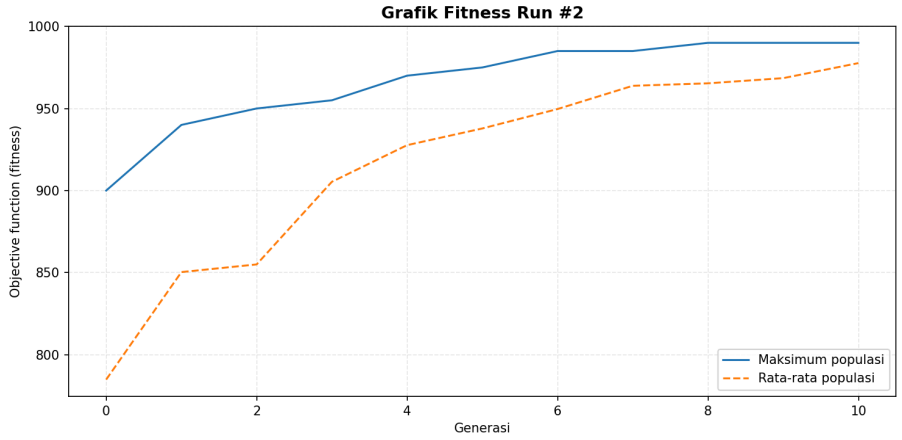
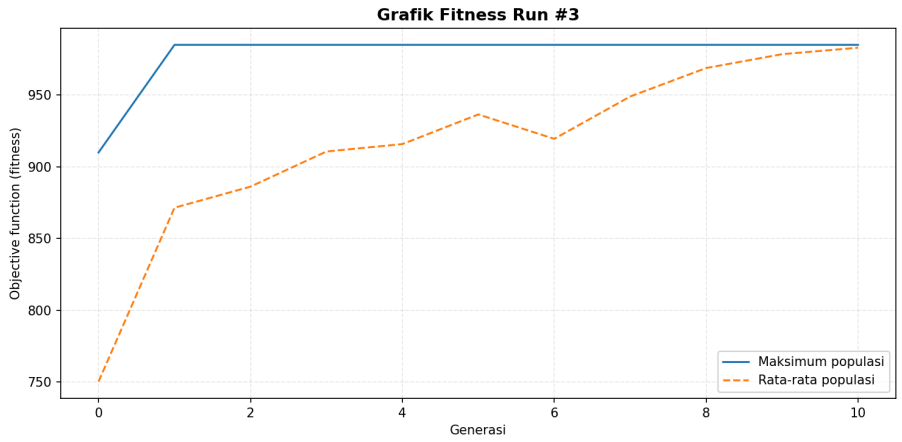


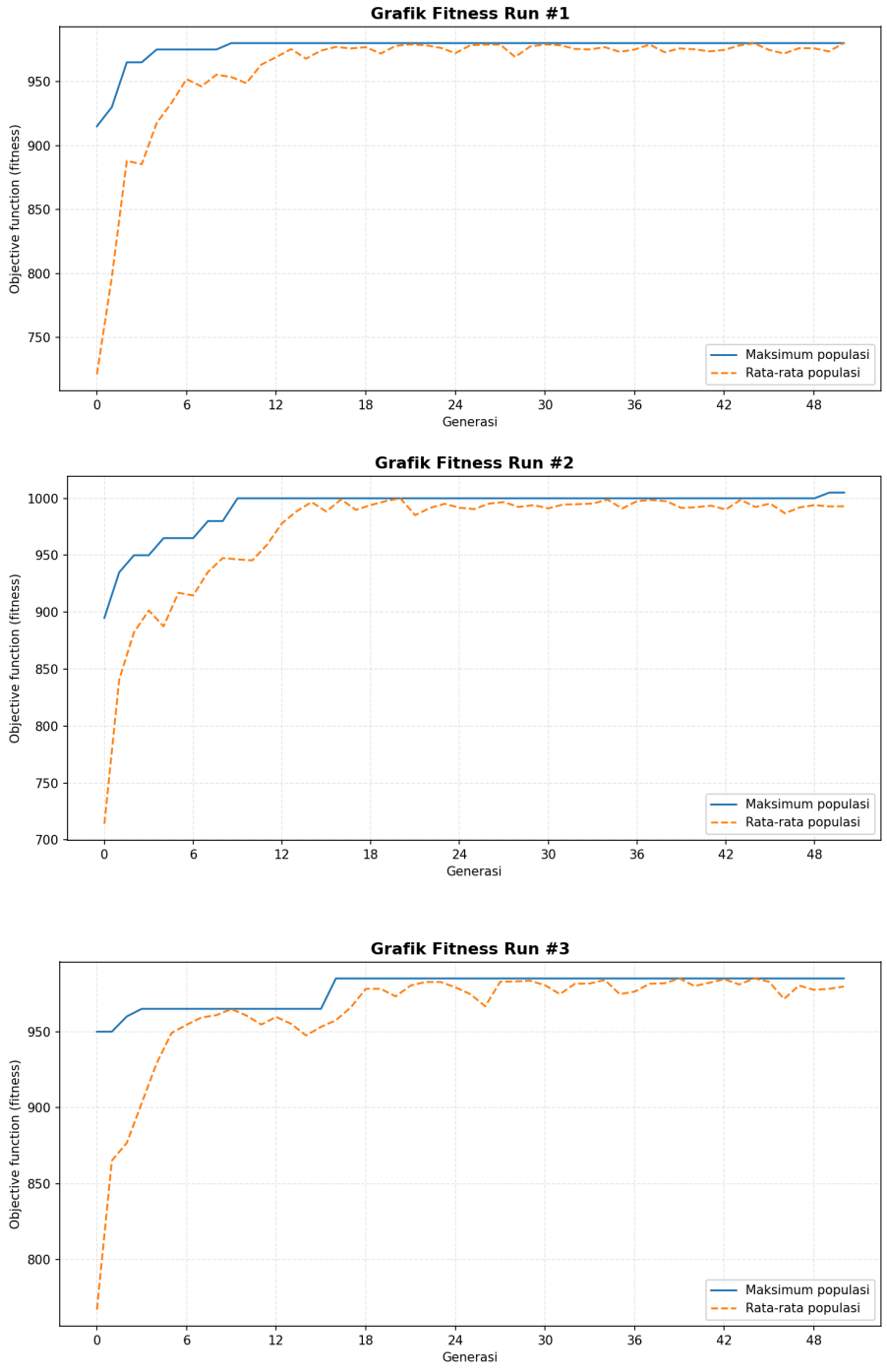


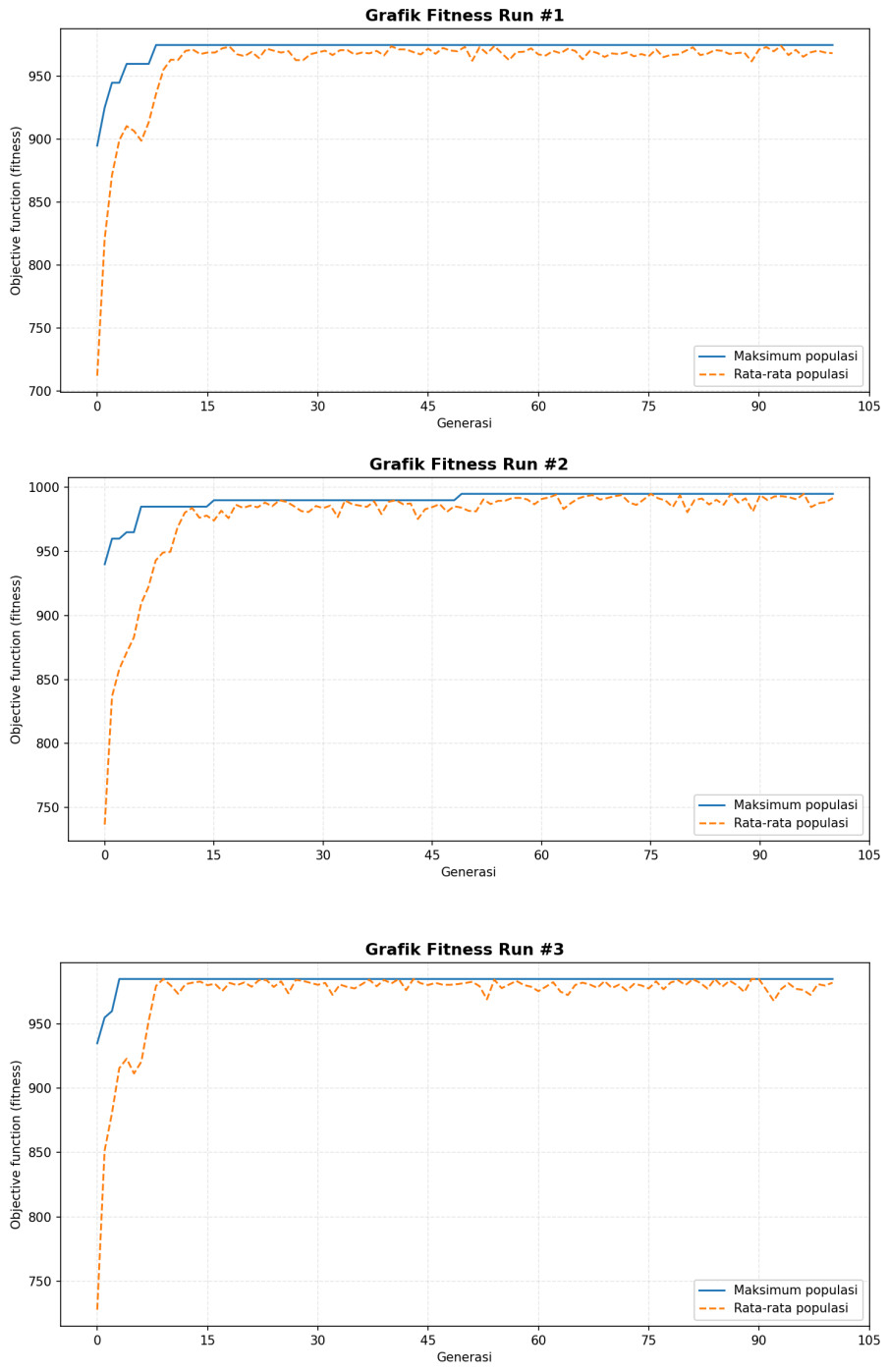


Di luar kapal: V01, V06, V07, V12, V14, V17, V18

Plot nilai objective function (maksimum dan rata-rata populasi) terhadap banyak iterasi

Variasi	Grafik Maksimum dan Rata-rata Populasi terhadap Iterasi
Variasi 1	  

Variasi	Grafik Maksimum dan Rata-rata Populasi terhadap Iterasi
Variasi 2	

Variasi	Grafik Maksimum dan Rata-rata Populasi terhadap Iterasi
Variasi 3	

Ringkasan hasil percobaan

Variasi	Percobaan	Jumlah Populasi	Banyak Iterasi	Initial Value	Final Value	Durasi (s)
Variasi 1	Percobaan 1	30	10	885	985	0,0566
	Percobaan 2	30	10	900	990	0,0454
	Percobaan 3	30	10	910	985	0,0484
Variasi 2	Percobaan 1	30	50	915	980	0,1961
	Percobaan 2	30	50	895	1005	0,1399
	Percobaan 3	30	50	950	985	0,1603
Variasi 3	Percobaan 1	30	100	895	975	0,1628
	Percobaan 2	30	100	940	995	0,1977
	Percobaan 3	30	100	935	985	0,2875

Perbandingan Eksperimen 1

Generasi	Rata-Rata Final	Rentang Final	Rata-Rata Durasi
10 (K1)	986,7	985–990	0,0501 s
40 (K2)	990,0	980–1005	0,1654 s
100 (K3)	985,0	975–995	0,2160 s

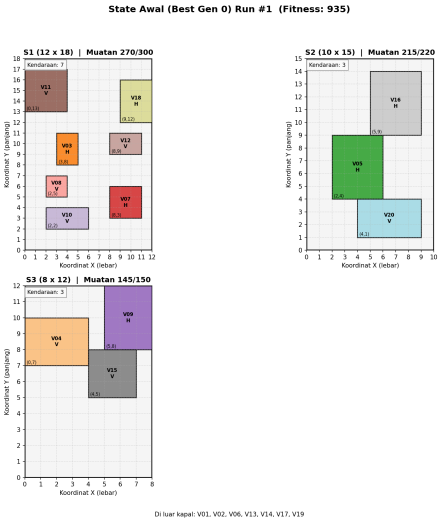
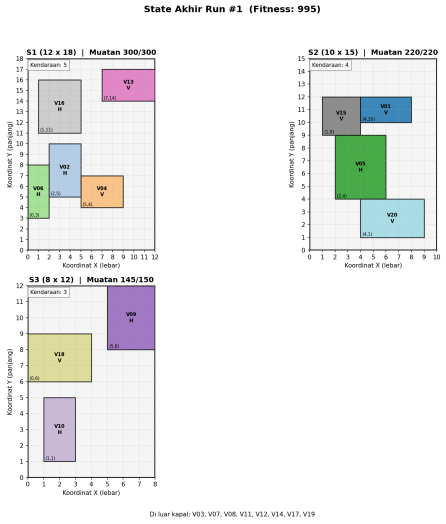
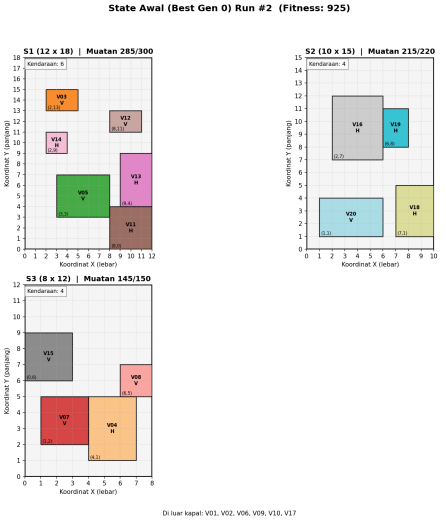
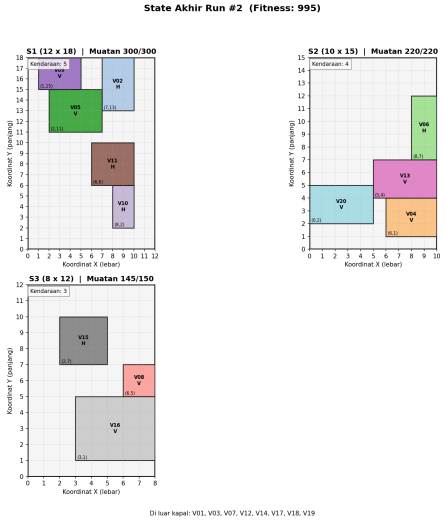
Pada Eksperimen 1, jumlah populasi dijaga tetap 30 dan banyak iterasi divariasikan menjadi 10, 50, dan 100. Rata-rata Final Value dari tiga percobaan adalah 986,7 pada 10 iterasi, 990,0 pada 50 iterasi, dan 985,0 pada 100 iterasi. Selisih antar variasi hanya 3 sampai 5 poin, sedangkan selisih antar percobaan dalam satu variasi dapat mencapai 25 poin (misalnya pada 50 iterasi, 980 sampai 1005). Dengan hanya tiga percobaan, perbedaan rata-rata tersebut belum cukup untuk menyimpulkan bahwa iterasi yang lebih banyak selalu memberi hasil lebih baik. Nilai terbaik yang dicapai adalah 1005 (Percobaan 2 pada 50 iterasi).

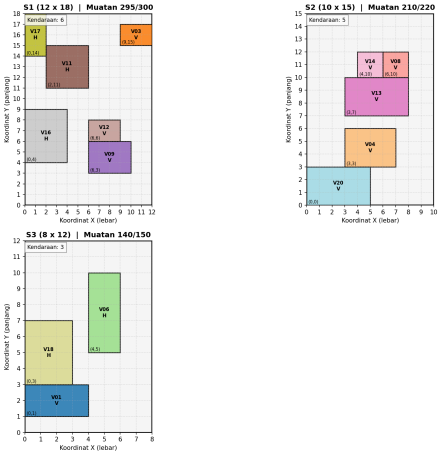
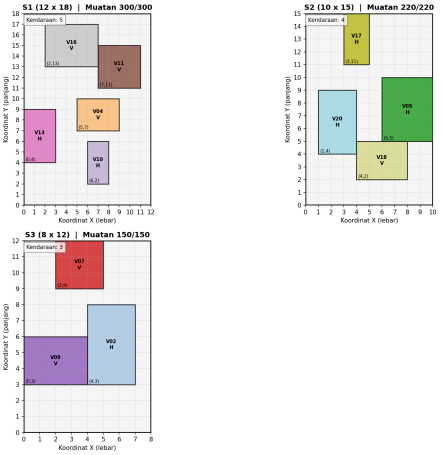
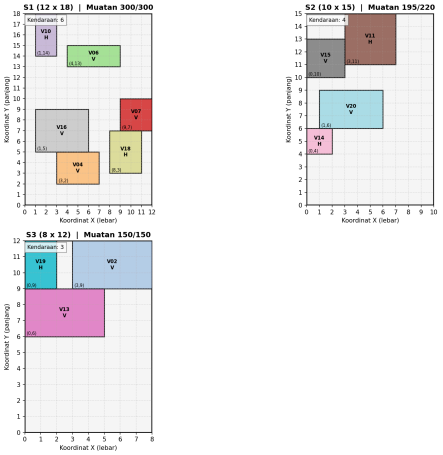
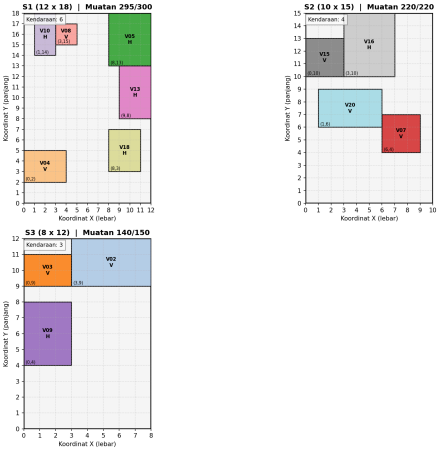
Dari grafik terlihat bahwa kenaikan nilai maksimum populasi terjadi paling banyak pada sekitar 10 iterasi pertama, lalu melambat. Pada beberapa percobaan nilai maksimum bertahan di nilai yang sama hingga akhir, misalnya 975 sejak iterasi ke-8 pada Percobaan 1 dengan 100 iterasi, yang menandakan populasi sudah konvergen dan kurang mampu menemukan perbaikan lebih lanjut. Rata-rata populasi yang naik mendekati nilai maksimum dan berfluktuasi kecil

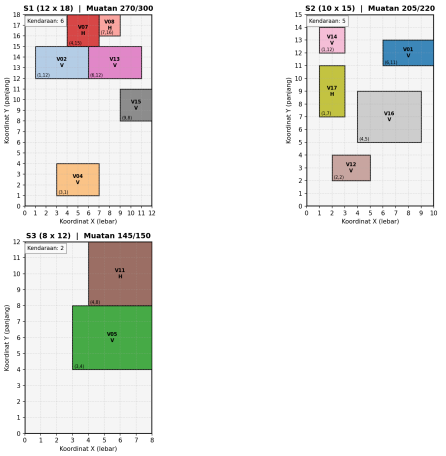
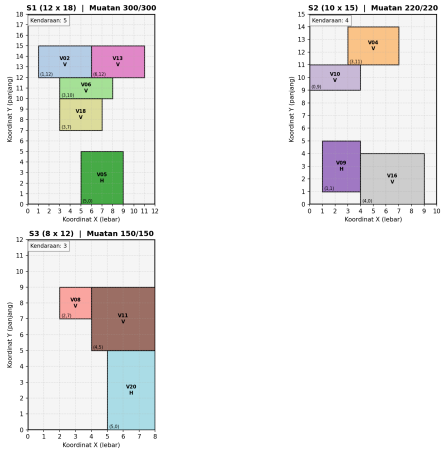
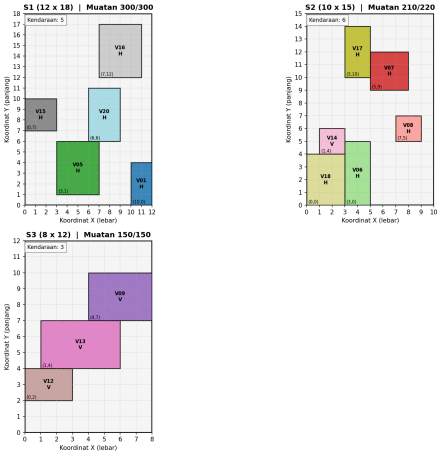
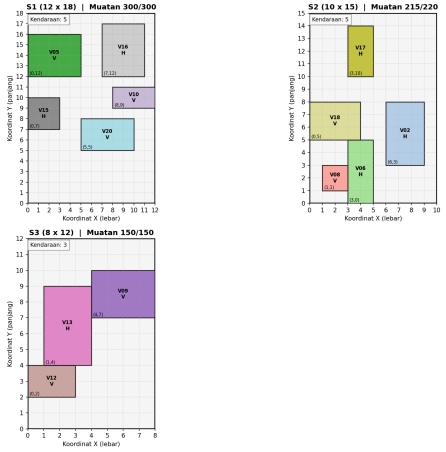
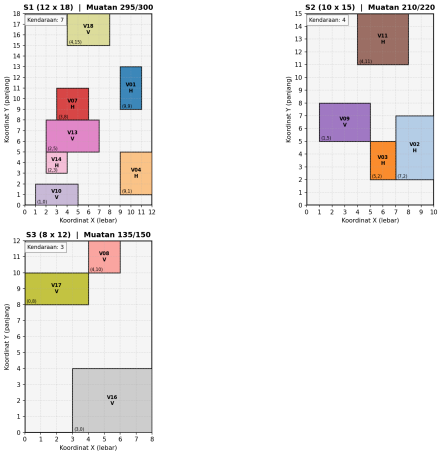
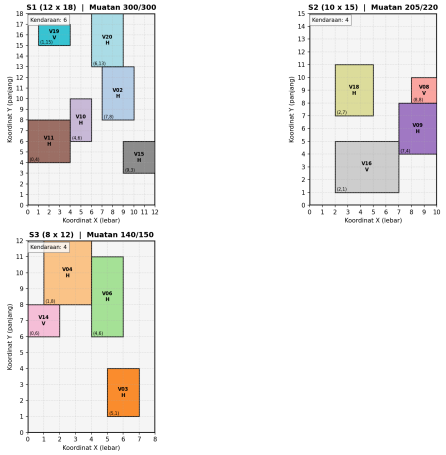
juga menunjukkan populasi mulai seragam. Durasi pencarian meningkat seiring banyaknya iterasi, dari rata-rata sekitar 0,05 detik (10 iterasi) menjadi sekitar 0,17 detik (50 iterasi) dan 0,22 detik (100 iterasi), namun seluruhnya masih di bawah satu detik.

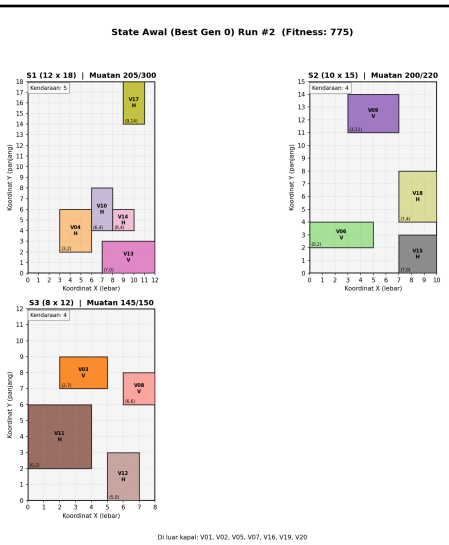
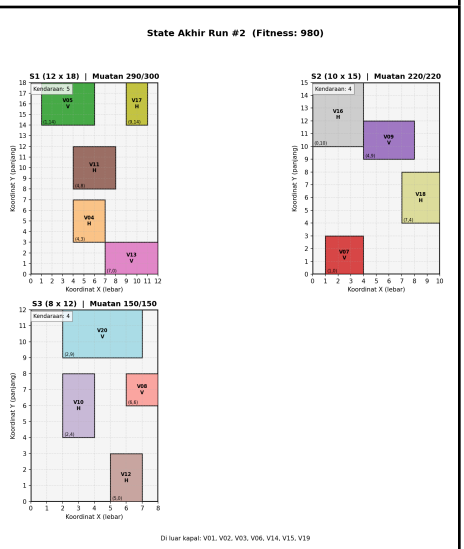
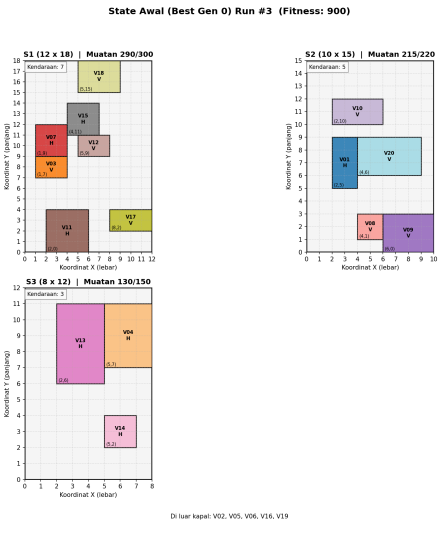
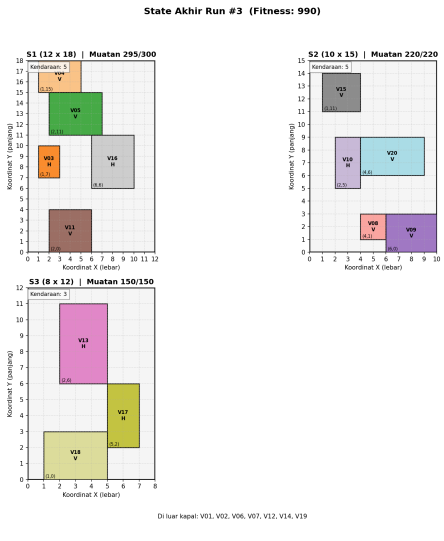
Eksperimen 2: banyak iterasi sebagai kontrol, 3 variasi jumlah populasi, masing-masing dijalankan 3 kali

State awal dan state akhir

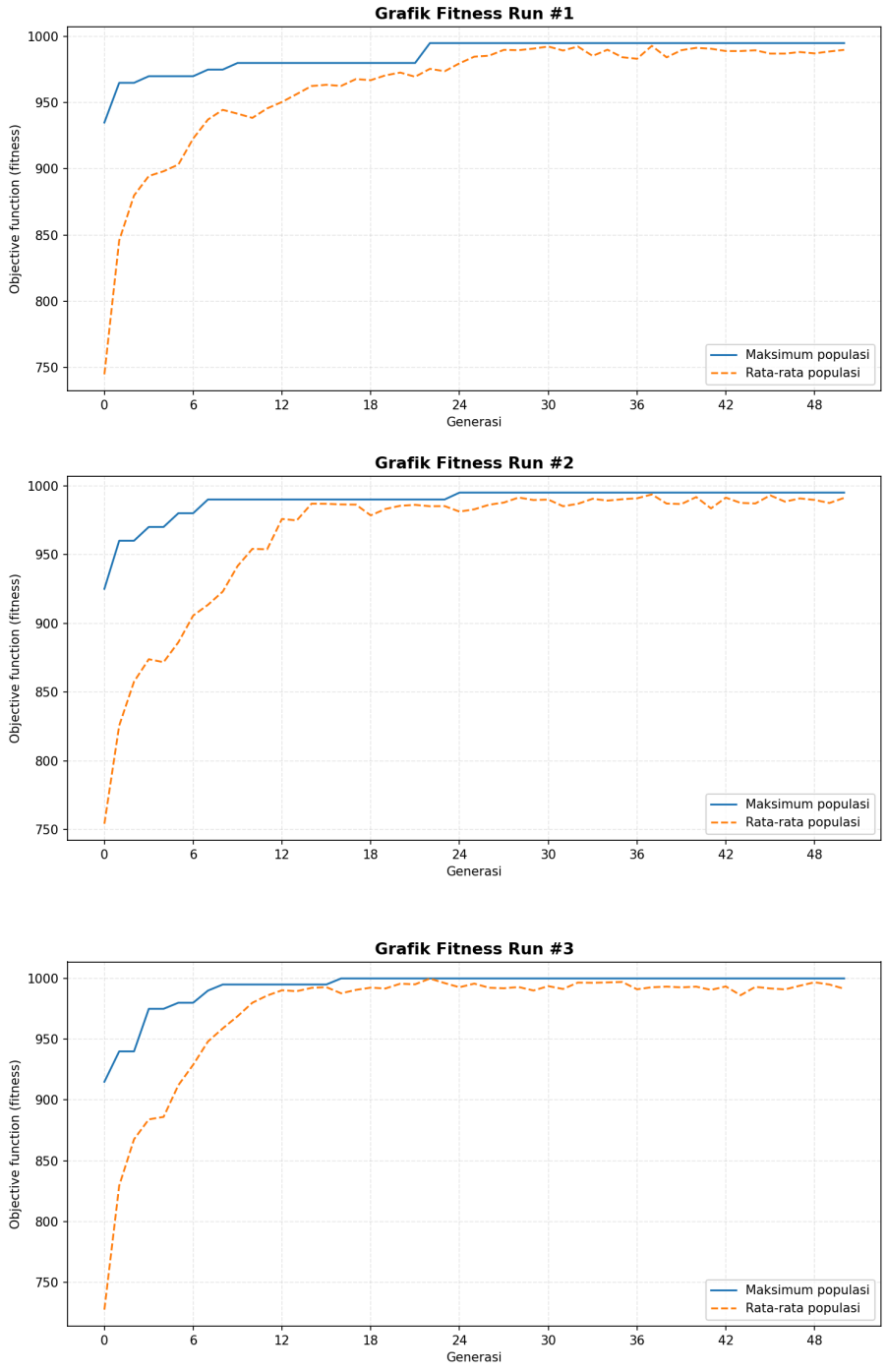
Variasi	State Awal	State Akhir
Variasi 1		
		

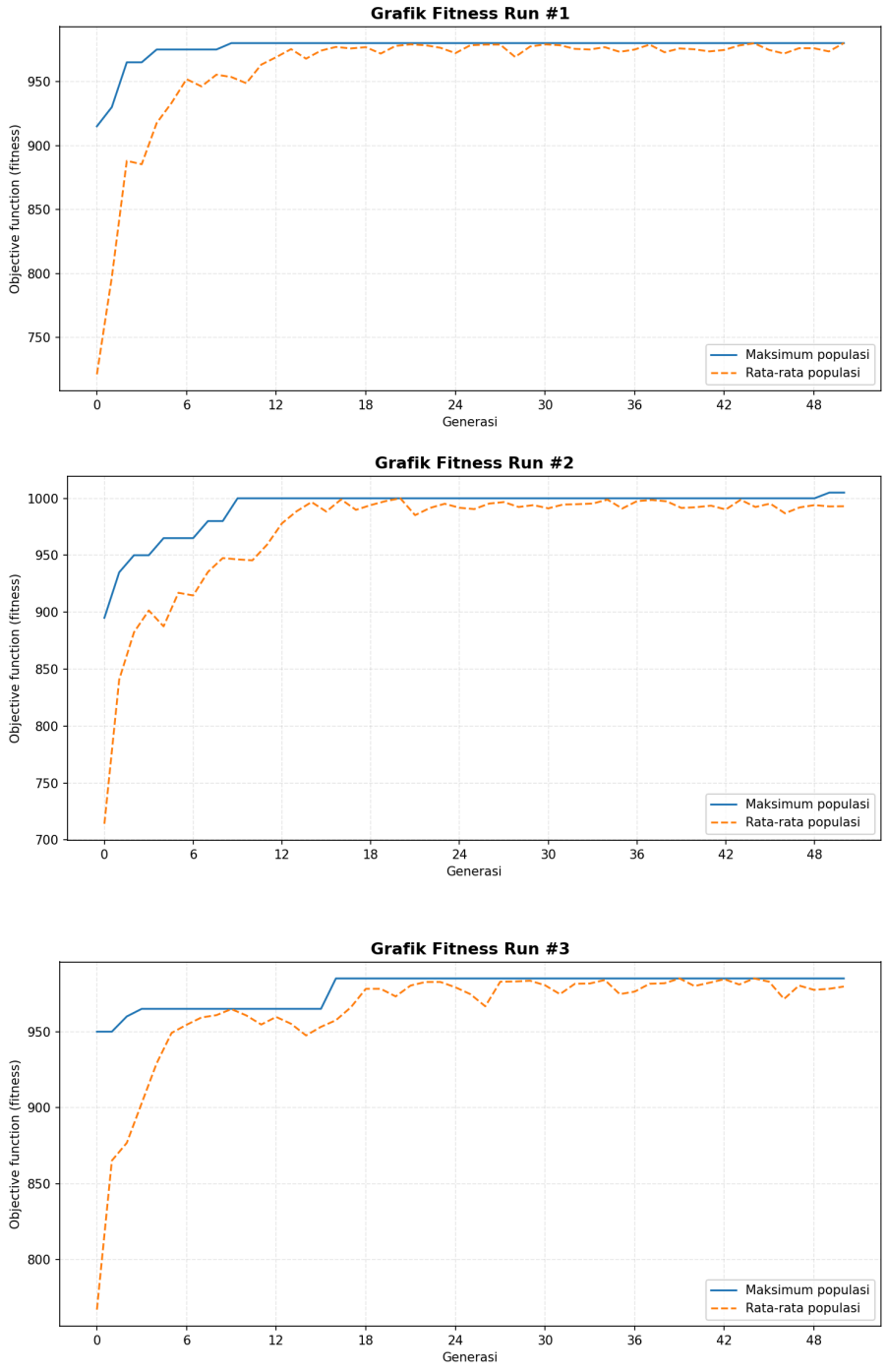
Variasi	State Awal	State Akhir
	State Awal (Best Gen 0) Run #3 (Fitness: 915)  Di luar kapal: V02, V05, V07, V10, V15, V19	State Akhir Run #3 (Fitness: 1000)  Di luar kapal: V01, V03, V06, V08, V12, V14, V15, V19
Variasi 2	State Awal (Best Gen 0) Run #1 (Fitness: 915)  Di luar kapal: V01, V03, V05, V08, V09, V12, V17	State Akhir Run #1 (Fitness: 980)  Di luar kapal: V01, V06, V11, V12, V14, V17, V19

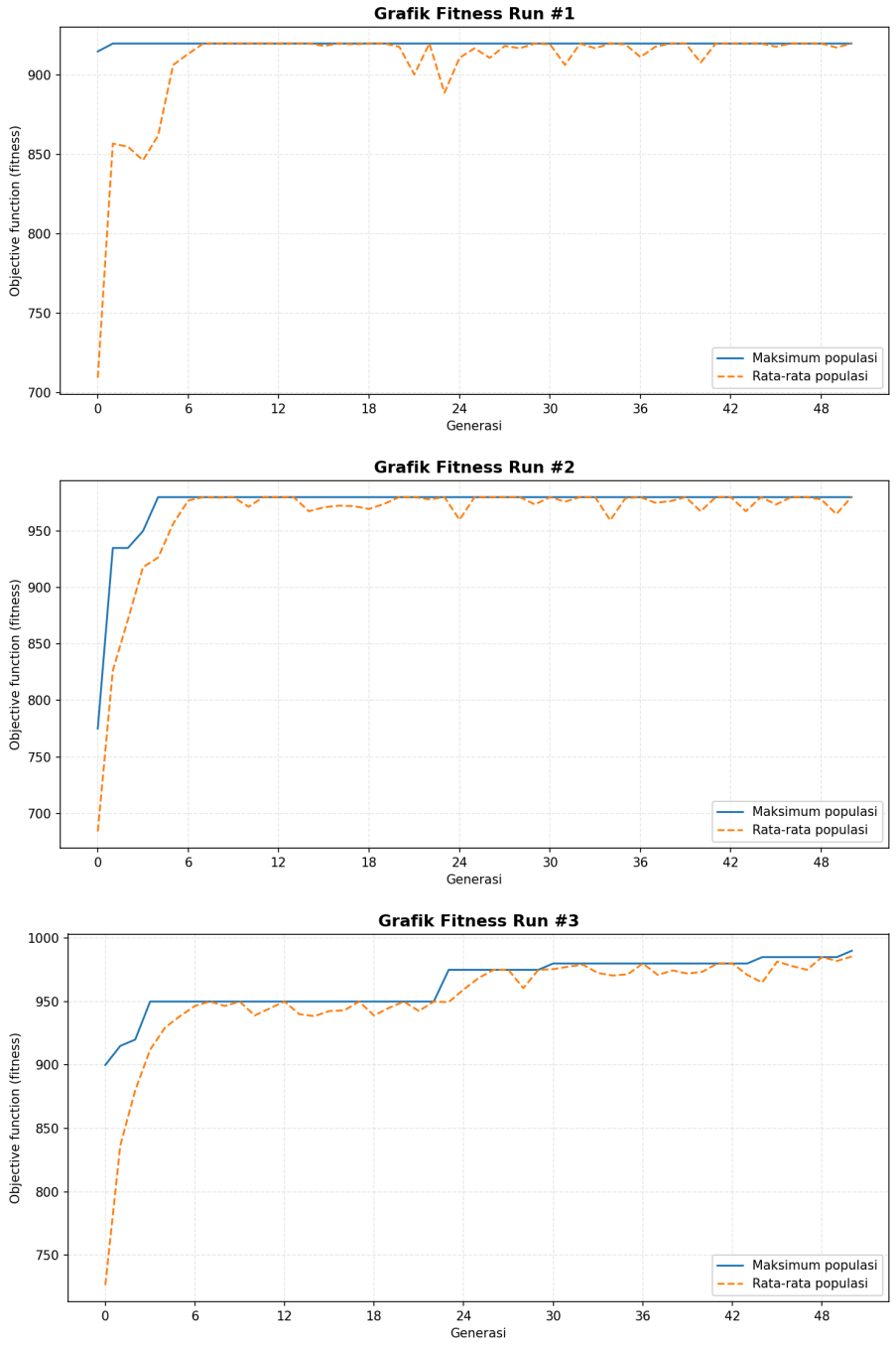
Variasi	State Awal	State Akhir
	State Awal (Best Gen 0) Run #2 (Fitness: 895)  Di luar kapal: V03, V06, V09, V10, V18, V19, V20	**State Akhir Run #2 (Fitness: 1005)**  Di luar kapal: V01, V03, V07, V12, V14, V15, V17, V19
	State Awal (Best Gen 0) Run #3 (Fitness: 950)  Di luar kapal: V02, V03, V04, V10, V11, V19	**State Akhir Run #3 (Fitness: 985)**  Di luar kapal: V01, V03, V04, V07, V11, V14, V19
Variasi 3	**State Awal (Best Gen 0) Run #1 (Fitness: 915)**  Di luar kapal: V05, V06, V12, V15, V19, V20	**State Akhir Run #1 (Fitness: 920)**  Di luar kapal: V01, V05, V07, V12, V13, V17

Variasi	State Awal	State Akhir
		
		

Plot nilai objective function (maksimum dan rata-rata populasi) terhadap banyak iterasi

Variasi	Grafik Maksimum dan Rata-rata Populasi terhadap Iterasi
Variasi 1	

Variasi	Grafik Maksimum dan Rata-rata Populasi terhadap Iterasi
Variasi 2	

Variasi	Grafik Maksimum dan Rata-rata Populasi terhadap Iterasi
Variasi 3	

Ringkasan hasil percobaan

Variasi	Percobaan	Jumlah Populasi	Banyak Iterasi	Initial Value	Final Value	Durasi (s)

Variasi 1	Percobaan 1	60	50	935	995	0,1869
	Percobaan 2	60	50	925	995	0,2125
	Percobaan 3	60	50	915	1000	0,1960
Variasi 2	Percobaan 1	30	50	915	980	0,1961
	Percobaan 2	30	50	895	1005	0,1399
	Percobaan 3	30	50	950	985	0,1603
Variasi 3	Percobaan 1	10	50	915	920	0,0350
	Percobaan 2	10	50	775	980	0,0323
	Percobaan 3	10	50	900	990	0,0531

Perbandingan Eksperimen 2

Populasi	**Rata-Rata Final**	**Rentang Final**	**Rata-Rata Durasi**
60	996,7	995–1000	0,1985 s
30	990,0	980–1005	0,1654 s
10	963,3	920–990	0,0401 s

Pada Eksperimen 2, banyak iterasi dijaga tetap 50 dan jumlah populasi divariasikan menjadi 10, 30, dan 60. Rata-rata Final Value meningkat seiring bertambahnya populasi, yaitu 963,3 (populasi 10), 990,0 (populasi 30), dan 996,7 (populasi 60). Selisih antar percobaan juga menyempit, dari 70 poin (920 sampai 990) pada populasi 10 menjadi 25 poin pada populasi 30 dan 5 poin (995 sampai 1000) pada populasi 60. Hal ini menunjukkan populasi yang lebih besar cenderung menghasilkan solusi yang lebih baik dan lebih konsisten, karena populasi awal dan hasil crossover mencakup lebih banyak kombinasi penempatan sehingga lebih kecil kemungkinan pencarian terjebak pada solusi yang kurang baik.

Populasi 10 menghasilkan hasil paling tidak stabil. Pada Percobaan 1, nilai maksimum hanya naik satu kali (915 ke 920) dan tidak berubah hingga iterasi ke-50, yang menandakan populasi cepat kehilangan keragaman. Sebaliknya, Percobaan 3 terus naik hingga iterasi terakhir (990), yang menunjukkan hasil populasi kecil sangat bergantung pada keberuntungan populasi awal dan mutasi. Durasi pencarian meningkat seiring populasi, dari rata-rata sekitar 0,04 detik

(populasi 10) menjadi 0,17 detik (populasi 30) dan 0,20 detik (populasi 60). Peningkatan durasi dari populasi 30 ke 60 relatif kecil dibanding peningkatan hasil. Nilai tertinggi keseluruhan (1005) justru dicapai pada populasi 30, sehingga dengan tiga percobaan, pengaruh populasi sebaiknya dipahami sebagai kecenderungan, bukan kepastian.

2.3.2. Hasil Analisis

1. Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?

Objective yang dipakai adalah total fee − 5 × total ETA. Untuk input2.json, total berat 20 kendaraan (925) melebihi total kapasitas kapal (670), sedangkan total luas kendaraan (215) jauh di bawah luas geladak (462). Jadi kapasitas berat adalah pembatas utama. Jika dihitung batas atas dengan pemilihan kendaraan yang optimal menurut kapasitas berat, tanpa memperhatikan geometri. Hasilnya 1005. Nilai 1005 pernah dicapai Genetic Algorithm, sehingga solusinya layak dan 1005 adalah global optima kasus ini.

Algoritma	Rata-Rata Final	Terbaik	Rasio terhadap 1005
Simple Hill Climbing	976,7	985	97,2%
Steepest Ascent	975,0	980	97,0%
Hill-Climbing with Sideways Move	968,3	980	96,4%
Random Restart Hill Climbing	1005,0	1005,0	100%
Simulated Annealing	935,0	980	93,0%
Genetic Algorithm (pop. 30, 50 iterasi)	990,0	1005	98,5%
Genetic Algorithm (pop. 60, 50 iterasi)	996,7	1000	99,2%

Untuk algoritma Simple Hill Climbing, Steepest Ascent, dan Sideways Move tidak ada yang mencapai global optima, sedangkan Random Restart Hill Climbing mencapainya pada ketiga percobaan (1005). Simulated Annealing

menghasilkan rata-rata nilai terendah. Pada percobaan ketiga, algoritma mencapai nilai 980 dalam 1838 iterasi, sesuai dengan jumlah iterasi yang diperlukan untuk menurunkan suhu dari 1000 menjadi 0,1 menggunakan cooling_rate sebesar 0,995. Sementara itu, percobaan pertama dan kedua berhenti lebih awal pada 506 dan 1139 iterasi, kemungkinan karena batas max_stagnation=500 tercapai sebelum proses pendinginan selesai. Genetic Algorithm menunjukkan hasil yang paling mendekati nilai optimal karena mampu mengeksplorasi banyak kombinasi solusi melalui populasi, crossover, dan mutasi. Bahkan, GA berhasil mencapai nilai 1005 pada percobaan kedua dengan populasi 30 dan 50 iterasi, meskipun keberhasilan tersebut hanya terjadi sekali dari tiga percobaan.

2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?

Urutan rata-rata final:

1. Random Restart
2. GA (populasi 60)
3. GA (populasi 30)
4. Simple Hill Climbing
5. Steepest Ascent
6. Sideways Move
7. Simulated Annealing.

Random Restart satu-satunya yang konsisten mencapai global optima. Selisih ketiga varian hill climbing lainnya (976,7; 975,0; 968,3) lebih kecil daripada variasi antar percobaannya sendiri (sekitar 9 sampai 13 poin), sehingga dengan tiga percobaan belum bisa dikatakan salah satunya lebih baik. Random Restart Hill Climbing, yang memiliki rata-rata 1005,0 dan rasio 100% dengan global optima dinyatakan algoritma yang terbaik untuk kasus ini. GA unggul sekitar 10 sampai 20 poin dari hill climbing dan lebih jauh dari Simulated Annealing, tapi dengan jumlah percobaan sedikit ini lebih tepat disebut kecenderungan daripada kepastian.

3. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?

Rata-rata durasi:

1. Simple HC 0,021 s
2. GA (populasi 10) 0,040 s
3. GA (populasi 30, 10 iterasi) 0,050 s
4. Steepest 0,125 s
5. GA (populasi 30, 50 iterasi) 0,165 s
6. GA (populasi 60) 0,199 s

7. Simulated Annealing 0,285 s
8. Sideways Move 2,361 s.
9. Random Restart 6,79 s

Random Restart memiliki durasi paling lambat dengan rata-rata 6,79 s. Sideways Move paling lambat kedua, sekitar 100 kali Simple HC, karena tiap iterasi membangkitkan dan menilai seluruh neighbor, dan jumlah iterasinya 35 sampai 37, jauh lebih banyak dari Steepest (4 sampai 7). Simple HC paling cepat karena menerima neighbor pertama yang lebih baik. Pada Simulated Annealing durasi tidak berbanding lurus dengan iterasi: 506 iterasi butuh 0,417 s, sedangkan 1139 dan 1838 iterasi hanya 0,206 s dan 0,231 s. Pengukuran ulang diperlukan sebelum menyimpulkan penyebabnya.

4. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?

Rentang final antar percobaan: Random Restart punya rentang 0 poin (1005 pada ketiganya), paling konsisten dari semua algoritma. Perlu dicatat bahwa initial value-nya berbeda jauh (600, 675, 755) tapi hasil akhirnya sama, jadi hasilnya tidak bergantung pada titik awal. Ini wajar, karena 50 restart menghapus ketergantungan itu. GA populasi 60 hanya 5 poin (995 sampai 1000), Steepest 15 poin, Simple HC 20, Sideways 25, GA populasi 30 sebesar 20 sampai 25, GA populasi 10 sebesar 70, dan Simulated Annealing paling lebar, 90 poin (890 sampai 980; simpangan baku 45). Hill climbing cukup konsisten karena semuanya naik ke area sekitar 965 sampai 985, tapi berhenti di titik berbeda sesuai initial state. Simulated Annealing paling tidak konsisten karena dua dari tiga percobaannya berhenti sebelum jadwal pendinginan selesai. Dengan tiga percobaan, simpangan ini masih kasar.

5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

Menaikkan iterasi dari 10 ke 50 ke 100 (populasi 30) tidak mengubah hasil secara berarti: rata-rata 986,7; 990,0; 985,0. Selisihnya ($\leq$5 poin) lebih kecil daripada variasi antar percobaan dalam satu konfigurasi (sampai 25 poin). Grafik menunjukkan kenaikan maksimum terjadi dalam sekitar 10 iterasi pertama lalu mendatar, tanda populasi sudah seragam. Jumlah populasi berpengaruh lebih jelas: rata-rata 963,3 (populasi 10), 990,0 (populasi 30), dan 996,7 (populasi 60), dengan rentang antar percobaan menyempit dari 70 ke 25 ke 5 poin. Populasi besar memberi keragaman awal lebih banyak, dengan biaya durasi yang naik dari 0,04 s ke 0,17 s ke 0,20 s.

3. KESIMPULAN DAN SARAN

3.1. Kesimpulan

Random Restart Hill Climbing mencapai global optima pada ketiga percobaan (rata-rata 1005,0). Genetic Algorithm menyusul dengan rata-rata 996,7 pada populasi 60 dan 990,0 pada populasi 30. Tiga varian hill climbing tanpa restart (Simple, Steepest Ascent, Sideways Move) berada di sekitar 968 sampai 977 dan tidak ada yang mencapai global optima. Simulated Annealing paling rendah dengan rata-rata 935,0. Selisih antar tiga varian hill climbing masih lebih kecil daripada variasi antar percobaannya, sehingga belum dapat dikatakan salah satunya lebih unggul.

Hill climbing tanpa restart berhenti pada local optimum atau area datar, dan batas 30 sideways move tidak cukup untuk keluar dari area datar karena seluruh percobaan berhenti akibat max_sideways_reached. Simulated Annealing pada dua dari tiga percobaan berhenti sebelum jadwal pendinginan selesai, kemungkinan karena max_stagnation=500. Hanya percobaan 3, yang menyelesaikan pendinginan penuh hingga 1838 iterasi, yang menghasilkan 980.

Algoritma tercepat adalah Simple Hill Climbing (rata-rata 0,021 s), sedangkan yang paling lambat adalah Random Restart (6,79 s), diikuti Sideways Move (2,36 s). Durasi Random Restart sama dengan jumlah restart dikali biaya satu kali Steepest Ascent. Dilihat dari mutu per waktu, Genetic Algorithm dengan populasi 60 paling efisien: rata-rata 996,7 dalam sekitar 0,20 s.

Konsistensi. Random Restart paling konsisten (rentang 0 poin), diikuti Genetic Algorithm populasi 60 (5 poin). Simulated Annealing paling tidak konsisten (rentang 90 poin).

Jumlah populasi pada algoritma GA berpengaruh lebih jelas daripada jumlah iterasi. Populasi 10, 30, dan 60 menghasilkan rata-rata 963,3; 990,0; dan 996,7 dengan rentang antar percobaan yang menyempit (70, 25, 5 poin), sedangkan iterasi 10, 50, dan 100 hanya menghasilkan perbedaan rata-rata di bawah 5 poin karena nilai maksimum populasi umumnya sudah mendatar setelah sekitar 10 iterasi pertama.

3.2. Saran

1. Perbanyak percobaan per konfigurasi (misalnya 10 atau lebih) agar perbedaan antar algoritma dapat diuji dengan simpangan baku yang lebih meyakinkan.

2. Stochastic Hill Climbing diuji pada input3.json dengan max_iter=100, sehingga tidak dapat dibandingkan. Ulangi pada input2.json, dan uji juga masukan yang lebih besar.
3. Coba max_sideways yang lebih besar (misalnya 100 sampai 500) dan catat apakah hasilnya mendekati 1005, karena batas 30 terbukti menghentikan semua percobaan.
4. Naikkan max_stagnation atau hapus syaratnya agar pendinginan selesai penuh, dan coba cooling_rate yang lebih lambat (misalnya 0,999). Ukur ulang durasi agar tidak tercampur gangguan sistem.
5. Catat nomor restart saat nilai terbaik pertama kali ditemukan. Jika 1005 sudah muncul sebelum restart ke-50, jumlah restart dapat dikurangi tanpa mengorbankan hasil.
6. Genetic Algorithm populasi 60 hampir menyamai Random Restart dengan durasi jauh lebih singkat. Menambahkan pencarian lokal (misalnya steepest ascent pada individu terbaik) ke dalam GA berpotensi menghasilkan 1005 secara konsisten dalam waktu singkat.

4. PEMBAGIAN TUGAS

NAMA	NIM	TUGAS
Indanna Halwah Raqiqah Masyhur	18224014	1. Mengimplementasikan algoritma *Hill-Climbing with Sideways Move* 2. Mengimplementasikan *Genetic Algorithm* 3. Menjalankan pengujian dan menulis bagian implementasi serta hasil eksperimen algoritma 4. Menyusun format, merapikan, serta menulis bagian umum laporan (deskripsi persoalan, dll.)
Evan Nathanael Tanuri	18224019	1. Mengimplementasikan semua varian *Hill-Climbing* selain *Sideways Move (Simple Hill Climbing, Steepest Ascent Hill Climbing, Random Restart Hill Climbing, Stochastic Hill Climbing)* 2. Menjalankan pengujian dan menulis bagian implementasi serta hasil eksperimen algoritma yang dikerjakan 3. Mendefinisikan *entity*, *state*, dan *objective function* proyek. 4. Bertanggung jawab atas alur program dan pengelolaan *repository* GitHub
Muhammad Okto Huzainy	18224021	1. Mengimplementasikan algoritma *Simulated Annealing* 2. Menjalankan pengujian dan menulis bagian implementasi serta hasil eksperimen algoritma yang dikerjakan 3. Membuat kode visualisasi

5. REFERENSI

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Sep. 16, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 16, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 16, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Sep. 16, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Sep. 16, 2026].

6. LAMPIRAN

https://github.com/evantanuri/IF3070_G10_Lampung-Cina.git